

ECE 218 — Exam 1 Practice Bank

45 extra exam-style questions in the instructor's style, beyond Sample1. Every trace, count, program output and compiler message was produced by actually compiling and running code, then independently re-checked.

Sorting (bubble, selection, insertion, merge, quick, generic C sort, templates)

S1 · trace

Bubble sort trace. The instructor's bubble sort (`sort.cpp`):

```
void bubbleSort(int *arr, const int n) {
    for(int i=0;i<n;i++) { // outer loop
        for(int j=0;j<n-1-i;j++) {
            if (arr[j]>arr[j+1]) {
                swap(arr[j],arr[j+1]);
            }
        }
    }
}
```

Sort 6 2 9 4 1 7 in ascending order. (a) Show every compare/swap of pass 1. (b) Show the array after each pass, marking the part that is already in its final position. (c) How many comparisons and swaps are made in total? (d) After which pass is the array sorted, and why does the code keep running?

Answer

(a) Pass 1 (i = 0, j = 0..4):

j	compare	action	array
0	6 > 2	swap	2 6 9 4 1 7
1	6 > 9?	no	2 6 9 4 1 7
2	9 > 4	swap	2 6 4 9 1 7
3	9 > 1	swap	2 6 4 1 9 7
4	9 > 7	swap	2 6 4 1 7 9

The largest value (9) has bubbled up to the last position.

(b) The | marks where the part in its final place begins.

pass	i	j runs	comps	swaps	array
1	0	0..4	5	4	2 6 4 1 7 9
2	1	0..3	4	2	2 4 1 6 7 9
3	2	0..2	3	1	2 1 4 6 7 9
4	3	0..1	2	1	1 2 4 6 7 9
5	4	0..0	1	0	1 2 4 6 7 9
6	5	none	0	0	1 2 4 6 7 9

(c) Comparisons = $5+4+3+2+1+0 = 15 = N(N-1)/2$ with $N = 6$. This is always 15 for $N = 6$, whatever the data. Swaps = **8**, which equals the number of inversions (pairs out of order): 6-2, 6-4, 6-1, 2-1, 9-4, 9-1, 9-7, 4-1.

(d) The array is sorted after **pass 4**. The code has no early-exit check, so passes 5 and 6 still run. Pass 5 makes 1 comparison. Pass 6 makes 0, because the inner loop condition becomes $j < 0$. After pass k the k largest values are in their final places, so pass k only needs to scan $a[0..N-1-k]$.

Verified: My own instrumented copy of the instructor's loops: `g++ -std=c++11 -Wall -Wextra -o v v.cpp` (in `verify_bank_sorting/`), then `./v bubble 6 2 9 4 1 7`:

```
j=0 ... swap => 2 6 9 4 1 7
j=1 ... no => 2 6 9 4 1 7
j=2 ... swap => 2 6 4 9 1 7
j=3 ... swap => 2 6 4 1 9 7
j=4 ... swap => 2 6 4 1 7 9
pass 1 comps=5 swaps=4 => 2 6 4 1 7 | 9
pass 2 comps=4 swaps=2 => 2 4 1 6 | 7 9
pass 3 comps=3 swaps=1 => 2 1 4 | 6 7 9
pass 4 comps=2 swaps=1 => 1 2 | 4 6 7 9
pass 5 comps=1 swaps=0 => 1 | 2 4 6 7 9
pass 6 comps=0 swaps=0
TOTAL passes=6 comps=15 swaps=8
```

Cross-check: `./trace bubble 6 2 9 4 1 7` (built from `tools/trace.cpp`) gives passes 2 6 4 1 7 9, 2 4 1 6 7 9, 2 1 4 6 7 9, 1 2 4 6 7 9, then two passes with no swaps, and `comparisons=15 swaps/moves=8`. The Python brute-force inversion count of `[6,2,9,4,1,7]` is 8.

S2 · write-code

Bubble sort with early exit. (a) Change the instructor's `bubbleSort` (`sort.cpp`) so that it stops as soon as a complete pass makes no swaps. (b) For each array, give the number of passes, comparisons and swaps of your version, and the comparisons of the original version: i) 6 1 2 3 4 5 ii) 2 3 4 5 6 1 iii) 1 2 3 4 5 6. (c) What is the best-case running time of each version, and for which input?

Answer

(a)

```
// bubble sort with early exit
// parameter: arr - int *: pointer to data array
// parameter: n - const int: number of elements in array
// return: none
void bubbleSortFlag(int *arr, const int n) {
    for (int i = 0; i < n; i++) {           // outer loop
        bool swapped = false;              // any swap in this pass?
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;               // no swaps -> already sorted -> stop
    }
}
```

(b)

array	early exit: passes / comps / swaps	original: comps
6 1 2 3 4 5	2 / 9 / 5	15
2 3 4 5 6 1	6 / 15 / 5	15
1 2 3 4 5 6	1 / 5 / 0	15

i) Pass 1 carries 6 all the way to the end (5 comps, 5 swaps), giving 1 2 3 4 5 6. Pass 2 (4 comps) makes no swap, so the sort stops. That is 9 comparisons instead of 15.

ii) The small value 1 moves only **one** place left per pass: 2 3 4 5 1 6, 2 3 4 1 5 6, 2 3 1 4 5 6, 2 1 3 4 5 6, 1 2 3 4 5 6. Passes 1-5 each make one swap. Pass 6 has 0 comparisons and no swap, so the sort stops (a loop written as `i < n-1` would stop after 5 passes). Total 15 comparisons: the flag saves nothing. Large values move right quickly; small values move left slowly.

iii) One pass of $N-1 = 5$ comparisons with no swaps, then stop.

(c) Original version: always $N(N-1)/2$ comparisons, even on sorted data, so it is $O(N^2)$ in the best, average and worst case.

Early-exit version: the best case is data that is already sorted. It takes one pass of $N-1$ comparisons, so it is $O(N)$. The worst case (reverse order) still takes $N(N-1)/2$ comparisons, which is $O(N^2)$.

Verified: `./v bflag 6 1 2 3 4 5` gives pass 1 comps=5 swaps=5 => 1 2 3 4 5 | 6, then pass 2 comps=4 swaps=0, (flag) no swaps -> stop, TOTAL passes=2 comps=9 swaps=5. `./v bubble 6 1 2 3 4 5` (no flag) gives 15 comparisons.

`./v bflag 2 3 4 5 6 1` gives passes 2 3 4 5 1 | 6, 2 3 4 1 | 5 6, 2 3 1 | 4 5 6, 2 1 | 3 4 5 6, 1 | 2 3 4 5 6 (1 swap each), then pass 6 comps=0 swaps=0, (flag) no swaps -> stop, TOTAL passes=6 comps=15 swaps=5.

`./v bflag 1 2 3 4 5 6` gives TOTAL passes=1 comps=5 swaps=0. `./v bflag 6 5 4 3 2 1` gives TOTAL passes=6 comps=15 swaps=15.

The code in (a) is copied verbatim into `algos.cpp`. `g++ -std=c++11 -Wall -Wextra -pedantic` gives no warnings. `valgrind -q --leak-check=full ./algos` prints `bubbleSortFlag: PASS (2000 arrays)` (random with duplicates, sorted and reverse inputs, checked against `std::sort`) and no valgrind errors.

S3 • trace

Insertion sort trace. Sort 7 3 9 2 6 1 in ascending order with insertion sort. The array has a sorted part and an unsorted part. Take the next element, move (shift) the larger sorted elements one place to the right, and drop the element into the hole. Count one comparison each time an element of the sorted part is compared with the element being inserted; reaching the front of the array is not a comparison. (a) For every step, show the element inserted, the comparisons, the shifts and the array. (b) Give the total comparisons and shifts. (c) Write the insertion sort function. (d) What are the best case and the worst case: which input, how many comparisons for $N = 6$, and what big-O?

Answer

(a)

step	next	comparisons made	comps	shifts	array (sorted unsorted)
1	3	7>3 move, reached front	1	1	3 7 9 2 6 1
2	9	7<=9 stop	1	0	3 7 9 2 6 1
3	2	9>2, 7>2, 3>2, reached front	3	3	2 3 7 9 6 1
4	6	9>6, 7>6, 3<=6 stop	3	2	2 3 6 7 9 1
5	1	9>1, 7>1, 6>1, 3>1, 2>1, reached front	5	5	1 2 3 6 7 9

(b) Comparisons = 1+1+3+3+5 = **13**. Shifts = 1+0+3+2+5 = **11**. The number of shifts always equals the number of inversions (pairs out of order) in the input, which is 11 here.

(c)

```
// insertion sort
// parameter: arr - int *: pointer to data array
// parameter: n - const int: number of elements in array
// return: none
void insertionSort(int *arr, const int n) {
    for (int i = 1; i < n; i++) {        // arr[0..i-1] is sorted
        int next = arr[i];              // next element to insert
        int j = i - 1;
        while (j >= 0 && arr[j] > next) { // move larger elements one to the right
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = next;              // drop it into the hole
    }
}
```

(d) Best case: the data is already sorted. Each new element is compared once and stays where it is, giving $N-1 = 5$ comparisons and 0 shifts, so **$O(N)$** .

Worst case: the data is in reverse order. Element i is compared with, and shifted past, all i sorted elements. That gives $1+2+\dots+(N-1) = N(N-1)/2 = 15$ comparisons and 15 shifts, so **$O(N^2)$** . The average case is $O(N^2)$.

Verified: `./v ins 7 3 9 2 6 1` (my own instrumented insertion sort; a comparison is counted only when $j \geq 0$):

```
step 1 next=3 comps=1 shifts=1 => 3 7 | 9 2 6 1
step 2 next=9 comps=1 shifts=0 => 3 7 9 | 2 6 1
step 3 next=2 comps=3 shifts=3 => 2 3 7 9 | 6 1
step 4 next=6 comps=3 shifts=2 => 2 3 6 7 9 | 1
step 5 next=1 comps=5 shifts=5 => 1 2 3 6 7 9
TOTAL comps=13 shifts=11
```

`./trace insertion 7 3 9 2 6 1` gives the same steps and comparisons=13 swaps/moves=11. The Python inversion count of [7,3,9,2,6,1] is 11.

`./v ins 1 2 3 4 5 6` gives TOTAL comps=5 shifts=0, and `./v ins 6 5 4 3 2 1` gives TOTAL comps=15 shifts=15.

The insertionSort in (c) is copied verbatim into `algos.cpp` (-Wall -Wextra -pedantic, no warnings):
insertionSort: PASS (2000 arrays), valgrind clean.

Selection sort (instructor version). The instructor's selection sort finds the **maximum** of the unsorted part and swaps it with the **last** unsorted element, so the sorted part grows from the right. (a) Show the array after each pass for 4 9 1 7 3 8. For each pass give the max found, its index, and the comparisons and swaps. (b) Give the total comparisons and swaps. (c) Write the function. (d) Why is selection sort $O(N^2)$ even when the data is already sorted?

Answer

(a) The | marks where the sorted part begins.

pass	unsorted part	max (index)	action	comps	array
1	a[0..5]	9 (1)	swap with a[5]=8	5	4 8 1 7 3 9
2	a[0..4]	8 (1)	swap with a[4]=3	4	4 3 1 7 8 9
3	a[0..3]	7 (3)	already last, no swap	3	4 3 1 7 8 9
4	a[0..2]	4 (0)	swap with a[2]=1	2	1 3 4 7 8 9
5	a[0..1]	3 (1)	already last, no swap	1	1 3 4 7 8 9
-	a[0]	-	one element left, so sorted	0	1 3 4 7 8 9

(b) Comparisons = $5+4+3+2+1 = 15 = N(N-1)/2$. Swaps = **3**. In passes 3 and 5 the max is already in the last unsorted slot, so nothing moves; the lecture counts these as 0 swaps.

If the swap is written without the `if`, it is still called in those passes and swaps an element with itself:

- With the loop `i < n-1` (as in (c)): 5 swap calls, 2 of them self-swaps.
- With the lecture's outer loop `0 -> N-1` (`i < N`): there is also a 6th pass on the single element `a[0]`, so 6 swap calls, 3 of them self-swaps.

For comparison, the textbook variant (find the min and swap it to the front) gives 1 9 4 7 3 8, 1 3 4 7 9 8, no swap, no swap, 1 3 4 7 8 9. It also makes 15 comparisons and 3 swaps.

(c)

```
// selection sort - find max, move it to the end of the unsorted part
// parameter: arr - int *: pointer to data array
// parameter: n - const int: number of elements in array
// return: none
void selectionSort(int *arr, const int n) {
    for (int i = 0; i < n - 1; i++) {
        int max_loc = 0;
        for (int j = 1; j < n - i; j++) { // unsorted part is arr[0..n-1-i]
            if (arr[j] > arr[max_loc]) max_loc = j;
        }
        if (max_loc != n - 1 - i) swap(arr[max_loc], arr[n - 1 - i]);
    }
}
```

(d) To find the maximum, it must look at every element of the unsorted part. It has no way to notice that the data is already in order. So it makes $(N-1)+(N-2)+\dots+1 = N(N-1)/2$ comparisons for any input: $O(N^2)$ in the

best, average and worst case. Only the number of swaps depends on the data. It is at most $N-1$, i.e. $O(N)$ swaps, which is selection sort's advantage.

Verified: `./v selmax 4 9 1 7 3 8` (find max, swap to end only if needed):

```
pass 1 max=9 (idx 1) => 4 8 1 7 3 9
pass 2 max=8 (idx 1) => 4 3 1 7 8 9
pass 3 max=7 (idx 3) => 4 3 1 7 8 9
pass 4 max=4 (idx 0) => 1 3 4 7 8 9
pass 5 max=3 (idx 1) => 1 3 4 7 8 9
TOTAL comps=15 swap-calls=3 self=0
```

Unconditional swap variants:

- `./v selmax_uncond_n1 4 9 1 7 3 8` ($i < n-1$): TOTAL comps=15 swap-calls=5 self=2
- `./v selmax_uncond_n 4 9 1 7 3 8` ($i < N$, as on the 8/27 page): TOTAL comps=15 swap-calls=6 self=3

`./trace selmax 4 9 1 7 3 8` gives the same passes and comparisons=15 swaps/moves=3. `./trace selmin 4 9 1 7 3 8` gives 1 9 4 7 3 8, 1 3 4 7 9 8, no swap, no swap, 1 3 4 7 8 9, with comparisons=15 swaps/moves=3.

The lecture example `./v selmax 5 4 8 1 3` gives 10 comparisons and 3 swaps, matching the 8/27 page (a self-swap counts as 0 swaps).

`./v selmax 1 2 3 4 5 6` gives TOTAL comps=15 swap-calls=0. The code in (c), in `algorithms.cpp`, prints `selectionSort: PASS (2000 arrays)` and sorts 4 9 1 7 3 8 to 1 3 4 7 8 9.

S5 · trace

Mergesort on an odd-length array. Use top-down mergesort with $\text{mid} = (\text{start} + \text{end}) / 2$, halves `[start..mid]` and `[mid+1..end]`, and merge with `if (A[i] < B[j])`. (a) Show the recursive splits and the merges for 7 2 9 4 1 8 5. (b) Give the comparisons of every merge and the total. (c) How many levels of splitting and how many merge operations are there for this $N = 7$ array? How many for $N = 1024$? What extra memory does mergesort need? (d) A student computes $\text{mid} = (\text{end} - \text{start} + 1) / 2 + \text{start}$ but keeps the halves `[start..mid]` and `[mid+1..end]`. What happens?

Answer

(a)/(b)

split	[7 2 9 4 1 8 5]	[0..6] -> [0..3] [4..6]	(left half gets the extra element)
split	[7 2 9 4]	[0..3] -> [0..1] [2..3]	
split	[7 2] -> [7] [2]	merge -> [2 7]	1 comp
split	[9 4] -> [9] [4]	merge -> [4 9]	1 comp
	merge [2 7] + [4 9]	-> [2 4 7 9]	3 comps
split	[1 8 5]	[4..6] -> [4..5] [6..6]	
split	[1 8] -> [1] [8]	merge -> [1 8]	1 comp
	merge [1 8] + [5]	-> [1 5 8]	2 comps
merge	[2 4 7 9] + [1 5 8]	-> [1 2 4 5 7 8 9]	6 comps

The last merge compares 2 vs 1 (take 1), 2 vs 5 (take 2), 4 vs 5 (take 4), 7 vs 5 (take 5), 7 vs 8 (take 7) and 9 vs 8 (take 8). The right half is then empty, so 9 is copied without a comparison. That is 6 comparisons.

Total = 1+1+3+1+2+6 = **14 comparisons**.

(c) For N = 7 there are **3 levels** of splitting (7 -> 4,3 -> 2,2,1 -> single elements), i.e. $\text{ceil}(\log_2 7) = 3$, and **6 merges**. Every merge turns two pieces into one, so there are always N-1 merges.

For N = 1024 there are $\log_2 1024 =$ **10 levels** and **1023 merges**. The merges on one level together handle all N elements, so the work is about $N \times \log_2 N$ whatever the data.

Extra memory: the merge needs a **work array** of N elements, i.e. $O(N)$, plus the $O(\log N)$ recursion stack.

(d) Take a 2-element range such as [0..1]. Then $\text{mid} = (1-0+1)/2 + 0 = 1 = \text{end}$, so the left call is mergesort(0,1) again. The range never gets smaller, which causes **infinite recursion** and a stack overflow (crash).

Fix: use $\text{mid} = (\text{start}+\text{end})/2$ with [start..mid] and [mid+1..end]. Or keep that formula and split into [start..mid-1] and [mid..end].

Verified: `./v msort 7 2 9 4 1 8 5` (my own top-down mergesort, $\text{mid}=(s+e)/2$, merge with <):

```
split [0..6] 7 2 9 4 1 8 5 -> [0..3] [4..6]
  split [0..3] 7 2 9 4 -> [0..1] [2..3]
    split [0..1] 7 2 -> [0..0] [1..1]
      merge 7 + 2 => 2 7 (1 comps)
    split [2..3] 9 4 -> [2..2] [3..3]
      merge 9 + 4 => 4 9 (1 comps)
    merge 2 7 + 4 9 => 2 4 7 9 (3 comps)
  split [4..6] 1 8 5 -> [4..5] [6..6]
    split [4..5] 1 8 -> [4..4] [5..5]
      merge 1 + 8 => 1 8 (1 comps)
    merge 1 8 + 5 => 1 5 8 (2 comps)
  merge 2 4 7 9 + 1 5 8 => 1 2 4 5 7 8 9 (6 comps)
result 1 2 4 5 7 8 9 comps=14 merges=6 maxdepth(split levels)=3
```

`./trace merge 7 2 9 4 1 8 5` gives the same tree and comparisons=14. `./v msort $(seq 1024 -1 1)` gives comps=5120 merges=1023 maxdepth(split levels)=10.

(d) midbug.cpp (g++ -std=c++11 -Wall -Wextra):

```
range [0..6] mid=3 -> [0..3] [4..6]
range [0..3] mid=2 -> [0..2] [3..3]
range [0..2] mid=1 -> [0..1] [2..2]
depth>1000 at range [0..1] -> infinite recursion
fixed version on [0..6] terminates, calls=13
```

S6 · trace

The merge step. Merge the sorted arrays A = 2 5 8 11 and B = 3 4 9 15 20 into C with the instructor's merge:

```
i=0, j=0, k=0
while (i < len(A) && j < len(B))
  if A[i] < B[j]: C[k]=A[i]; k++; i++
  else:          C[k]=B[j]; k++; j++
while (i < len(A)): C[k]=A[i]; k++; i++
while (j < len(B)): C[k]=B[j]; k++; j++
```


(a) Show i, j, k and the value copied at every step, and count the comparisons. (b) What are the minimum and maximum numbers of comparisons when merging 4 elements with 5 elements? Give an example of each. (c) Write the merge as a C++ function.

Answer

(a)

i	j	k	compare	copy	C so far
0	0	0	$2 < 3$	$C[0]=A[0]=2, i++$	2
1	0	1	$5 < 3$? no	$C[1]=B[0]=3, j++$	2 3
1	1	2	$5 < 4$? no	$C[2]=B[1]=4, j++$	2 3 4
1	2	3	$5 < 9$	$C[3]=A[1]=5, i++$	2 3 4 5
2	2	4	$8 < 9$	$C[4]=A[2]=8, i++$	2 3 4 5 8
3	2	5	$11 < 9$? no	$C[5]=B[2]=9, j++$	2 3 4 5 8 9
3	3	6	$11 < 15$	$C[6]=A[3]=11, i++$	2 3 4 5 8 9 11
4	3	7	A done (i = 4)	leftover $C[7]=B[3]=15$	... 15
4	4	8	-	leftover $C[8]=B[4]=20$	2 3 4 5 8 9 11 15 20

There are **7 comparisons**: 9 elements, of which the last 2 are copied without comparing. In general, comparisons = $(nA + nB) - (\text{number of leftovers copied})$.

(b) Minimum = $\min(nA, nB) = 4$. Example: A = 1 2 3 4, B = 5 6 7 8 9. A empties after 4 comparisons, and then B is just copied.

Maximum = $nA + nB - 1 = 8$. Example: A = 2 4 6 8, B = 1 3 5 7 9 (interleaved). Only the last element (9) is copied without a comparison.

Either way, merging is $O(nA + nB) = O(N)$.

(c)

```
// merge sorted A (na elements) and sorted B (nb elements) into C (na+nb elements)
// parameter: A, B - const int *: sorted input arrays
// parameter: na, nb - const int: number of elements in A and B
// parameter: C - int *: output array, must hold na+nb elements
// return: number of comparisons made
int merge(const int *A, const int na, const int *B, const int nb, int *C) {
    int i = 0, j = 0, k = 0, comps = 0;
    while (i < na && j < nb) {
        comps++;
        if (A[i] < B[j]) { C[k] = A[i]; k++; i++; }
        else { C[k] = B[j]; k++; j++; }
    }
    while (i < na) { C[k] = A[i]; k++; i++; } // leftovers of A
    while (j < nb) { C[k] = B[j]; k++; j++; } // leftovers of B
    return comps;
}
```

Verified: ./v mergeab "2 5 8 11" "3 4 9 15 20":

```

i=0 j=0 k=0 2<3 take A -> C[0]=2
i=1 j=0 k=1 5>=3 take B -> C[1]=3
i=1 j=1 k=2 5>=4 take B -> C[2]=4
i=1 j=2 k=3 5<9 take A -> C[3]=5
i=2 j=2 k=4 8<9 take A -> C[4]=8
i=3 j=2 k=5 11>=9 take B -> C[5]=9
i=3 j=3 k=6 11<15 take A -> C[6]=11
leftover B C[7]=15
leftover B C[8]=20
C = 2 3 4 5 8 9 11 15 20 comps=7

```

`./v mergeab "1 2 3 4" "5 6 7 8 9"` gives `comps=4`, and `./v mergeab "2 4 6 8" "1 3 5 7 9"` gives `comps=8`.

The function in (c) is copied verbatim into `mergefn.cpp` (`g++ -std=c++11 -Wall -Wextra -pedantic`; the only warning is in my test loop, not in the function). `valgrind -q ./mergefn` prints `2 3 4 5 8 9 11 15 20 comps=7` and PASS 5000 random merges (checked against `std::merge`), with no valgrind errors.

S7 · trace

Lomuto partition trace. `partition1` from `sortp.cpp`:

```

int partition1(int *a, const int start, const int end) {
    int pivot = a[end];
    int i = start-1;
    for (int j=start; j<end; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i+1], a[end]); // move end value to correct location
    return i+1;
}

```

Trace `partition1(a, 0, 6)` for `a = 7 2 9 1 6 4 5`. Show `i, j` and every swap.

- What is returned, and what is the array afterwards?
- What is true about the two sides and about the pivot?
- Which calls does `quicks` make next?

Answer

`pivot = a[6] = 5`, `i = -1`.

j	a[j]	a[j] <= 5 ?	i	swap	array
0	7	no	-1	-	7 2 9 1 6 4 5
1	2	yes	0	a[0]<->a[1]	2 7 9 1 6 4 5
2	9	no	0	-	2 7 9 1 6 4 5
3	1	yes	1	a[1]<->a[3]	2 1 9 7 6 4 5
4	6	no	1	-	2 1 9 7 6 4 5
5	4	yes	2	a[2]<->a[5]	2 1 4 7 6 9 5
end				a[i+1]=a[3] <-> a[6]	2 1 4 5 6 9 7

It **returns 3**, and the array is 2 1 4 5 6 9 7. That took 6 comparisons (end-start) and 4 swaps.

- i is the slow index: a[start..i] holds the values <= pivot found so far. j is the fast index that scans.
- After the partition, a[0..2] = 2 1 4 are all <= 5 and a[4..6] = 6 9 7 are all > 5.
- The pivot 5 is at index 3, which is **its final sorted position** (sorted order: 1 2 4 5 6 7 9).
- Next come quicks(a, 0, 2) and quicks(a, 4, 6), i.e. p-1 and p+1. The pivot is never touched again.
- When i == j the code swaps an element with itself, which is harmless. For example, partitioning 8 9 swaps a[0] with a[0] and a[1] with a[1].

Verified: ./v lomuto 7 2 9 1 6 4 5 (partition1 copied from sortp.cpp, with counters added):

```

pivot=a[6]=5 i=-1
j=0 a[j]=7 > pivot, skip (i=-1)
j=1 a[j]=2 <= pivot -> i=0 swap(a[0],a[1]) => 2 7 9 1 6 4 5
j=2 a[j]=9 > pivot, skip (i=0)
j=3 a[j]=1 <= pivot -> i=1 swap(a[1],a[3]) => 2 1 9 7 6 4 5
j=4 a[j]=6 > pivot, skip (i=1)
j=5 a[j]=4 <= pivot -> i=2 swap(a[2],a[5]) => 2 1 4 7 6 9 5
end swap(a[3],a[6]) => 2 1 4 5 6 9 7 return 3
comps=6 swapcalls=4 self=0

```

./trace lomuto 7 2 9 1 6 4 5 gives p=3 => 2 1 4 5 6 9 7 and comparisons=6 swaps/moves=4.

./v lomuto 8 9 gives swap(a[0],a[0]), then end swap(a[1],a[1]) => 8 9 return 1, and swapcalls=2 self=2.

S8 · trace

Hoare partition trace. partition2 from sortp.cpp:

```

int partition2(int *a, const int start, const int end) {
    int pivot = a[start];
    int i = start-1;
    int j = end+1;
    while (1) {
        i++;
        while (a[i]<pivot) i++;
        j--;
        while (a[j]>pivot) j--;
        if (i>=j) return j;
    }
}

```

```

    swap(a[i],a[j]);
}
}

```

(a) Trace `partition2(a, 0, 6)` for `a = 5 9 2 7 1 8 3`. Show where `i` and `j` stop in each round, the swaps, the returned value and the final array. (b) Is the pivot in its final position? (c) How must quicksort recurse when it uses `partition2`? What happens if `sortp.cpp`'s `quicks`, which recurses on `start..p-1` and `p+1..end`, simply calls `partition2` instead of `partition1`?

Answer

(a) pivot = `a[0] = 5`, `i = -1`, `j = 7`.

round	i moves	j moves	action	array
1	<code>i=0: 5<5? no, so i=0</code>	<code>j=6: 3>5? no, so j=6</code>	<code>0<6: swap a[0]<->a[6]</code>	3 9 2 7 1 8 5
2	<code>i=1: 9<5? no, so i=1</code>	<code>j=5: 8>5 yes; j=4: 1>5? no, so j=4</code>	<code>1<4: swap a[1]<->a[4]</code>	3 1 2 7 9 8 5
3	<code>i=2: 2<5 yes; i=3: 7<5? no, so i=3</code>	<code>j=3: 7>5 yes; j=2: 2>5? no, so j=2</code>	<code>i=3 >= j=2: return 2</code>	3 1 2 7 9 8 5

It returns **2**. The result is `3 1 2 | 7 9 8 5`: `a[0..2]` are all `<= 5` and `a[3..6]` are all `>= 5`. There were 2 swaps.

(b) **No**. The pivot 5 ended at index 6, but its sorted position is index 3 (sorted order: 1 2 3 5 7 8 9). Hoare only guarantees that the left part `<= pivot <=` the right part. The returned `j` is just the boundary between the two parts.

(c) The recursion must include `p` in the left part:

```

void quicksH(int *a, const int start, const int end) {
    if (start<end) {
        int p = partition2(a,start,end);
        quicksH(a,start,p);      // NOT p-1
        quicksH(a,p+1,end);
    }
}

```

With `quicks(a,start,p-1); quicks(a,p+1,end);`, the element at index `p` is left out of both recursive calls. That is only correct if `a[p]` is already in its final place, which Hoare does not guarantee. So the result can be wrong: for `5 9 2 7 1 8 3` it produces `1 3 2 5 7 8 9`, while the correct recursion produces `1 2 3 5 7 8 9`. The `p-1 / p+1` recursion is only correct for Lomuto, which puts the pivot in its final place.

Verified: `./v hoare 5 9 2 7 1 8 3` (partition2 copied from `sortp.cpp`, with prints added):

```

pivot=a[0]=5
i stops 0 (5), j stops 6 (3) -> swap => 3 9 2 7 1 8 5
i stops 1 (9), j stops 4 (1) -> swap => 3 1 2 7 9 8 5
i stops 3 (7), j stops 2 (2) -> return 2  3 1 2 7 9 8 5
swaps=2

```

`./trace hoare 5 9 2 7 1 8 3` gives returned `j=2 => 3 1 2 | 7 9 8 5`.

`./v hoarebug 5 9 2 7 1 8 3`:

```

p-1/p+1: 1 3 2 5 7 8 9
p/p+1:   1 2 3 5 7 8 9

```

qh.cpp contains the quicksH code in (c) verbatim. It prints 1 2 3 5 7 8 9 <- start..p and 1 3 2 5 7 8 9 <- start..p-1, then quicksH failures 0/3000; p-1 version failures 2272/3000 on random arrays checked against std::sort.

S9 · trace

Quicksort recursion tree. Use the instructor's quicksort from sortp.cpp:

```
void quicks(int *a, const int start, const int end) {
    if (start < end) {
        int p = partition1(a, start, end);    // Lomuto, pivot = a[end]
        quicks(a, start, p-1);
        quicks(a, p+1, end);
    }
}
```

Sort 6 3 8 1 9 2 5.

- Draw the recursion tree. For every call give (start,end), the pivot, the returned p and the array after the partition.
- Show the steps of the first partition.
- How many comparisons and calls are made in total?

Answer

First partition, quicks(0,6), with pivot = 5 and i = -1:

- j=0: 6 > 5, skip.
- j=1: 3 <= 5, so i=0 and swap a0,a1, giving 3 6 8 1 9 2 5.
- j=2: 8 > 5, skip.
- j=3: 1 <= 5, so i=1 and swap a1,a3, giving 3 1 8 6 9 2 5.
- j=4: 9 > 5, skip.
- j=5: 2 <= 5, so i=2 and swap a2,a5, giving 3 1 2 6 9 8 5.
- End: swap a3,a6, giving 3 1 2 5 9 8 6, and p = 3.

```
quicks(0,6) pivot 5 -> p=3 : 3 1 2 [5] 9 8 6      6 comps
|-- quicks(0,2) pivot 2 -> p=1 : 1 [2] 3          2 comps
|  |-- quicks(0,0) 1 element (stop)
|  |-- quicks(2,2) 1 element (stop)
|-- quicks(4,6) pivot 6 -> p=4 : [6] 8 9          2 comps
    |-- quicks(4,3) empty (stop)
    |-- quicks(5,6) pivot 9 -> p=6 : 8 [9]          1 comp
        |-- quicks(5,5) 1 element (stop)
        |-- quicks(7,6) empty (stop)
```

The result is 1 2 3 5 6 8 9. Comparisons = 6+2+2+1 = **11**. There are **9 calls**: 4 that partition and 5 base cases where start >= end.

Observations:

- The first pivot, 5, happened to be the true median, so the split was balanced (3 | 3). That is best-case behaviour, $O(N \log N)$.

- In [4..6] the pivot 6 was the smallest value, so the split was 0 | 2. That is worst-case behaviour: if it happens at every level, the total is $N(N-1)/2$, i.e. $O(N^2)$.

Verified: ./v quick 6 3 8 1 9 2 5 (my own instrumented quicks + partition1):

```
quicks(0,6) pivot=5 p=3 => 3 1 2 5 9 8 6 (6 comps)
  quicks(0,2) pivot=2 p=1 => 1 2 3 (2 comps)
    quicks(0,0) base
    quicks(2,2) base
  quicks(4,6) pivot=6 p=4 => 6 8 9 (2 comps)
    quicks(4,3) base
    quicks(5,6) pivot=9 p=6 => 8 9 (1 comps)
      quicks(5,5) base
      quicks(7,6) base
result 1 2 3 5 6 8 9 comps=11 calls=9 swapcalls=9 self=2
```

First partition, from ./v lomuto 6 3 8 1 9 2 5:

- $j=1$ $a[j] \leq \text{pivot} \rightarrow i=0$ swap($a[0], a[1]$) => 3 6 8 1 9 2 5
- $j=3$... swap($a[1], a[3]$) => 3 1 8 6 9 2 5
- $j=5$... swap($a[2], a[5]$) => 3 1 2 6 9 8 5
- end swap($a[3], a[6]$) => 3 1 2 5 9 8 6 return 3

./trace quick 6 3 8 1 9 2 5 shows the same 4 partitions and comparisons=11.

S10 · short-answer

Median-of-3 pivot. Median-of-3 looks at the first, middle ($\text{mid} = (\text{start} + \text{end})/2$) and last elements, and uses the median of the three as the pivot. (a) Give the pivot for: i) 8 3 6 1 9 2 5 ii) 1 2 3 4 5 6 7 8 9 iii) 9 8 7 6 5 4 3 2 1 iv) 20 11 3 17 25 9 14 6 v) 12 4 7 15 3 10. (b) For ii), the median is swapped into $a[\text{end}]$ and partition1 (Lomuto) is run. What is the result? What would happen with the plain pivot = $a[\text{end}]$? (c) Compare the number of comparisons of quicksort with pivot = last and with median-of-3 on sorted data with $N = 1000$. Does median-of-3 remove every $O(N^2)$ case?

Answer

(a)

array	first	middle (index)	last	median = pivot
i) 8 3 6 1 9 2 5	8	1 (3)	5	5
ii) 1 2 3 4 5 6 7 8 9	1	5 (4)	9	5
iii) 9 8 7 6 5 4 3 2 1	9	5 (4)	1	5
iv) 20 11 3 17 25 9 14 6	20	17 (3)	6	17
v) 12 4 7 15 3 10	12	7 (2)	10	10

For an even length, $\text{mid} = (0 + N - 1)/2$ is the lower middle: index 3 for $N=8$ and index 2 for $N=6$. This is the same as the lecture example 8 1 12 3 6 9, which uses 8, 12 (index 2) and 9, giving pivot 9.

(b) Swapping $a[4]=5$ with $a[8]=9$ gives 1 2 3 4 9 6 7 8 5. Partition1 with pivot 5 then gives 1 2 3 4 5 6 7 8 9 with $p = 4$, a perfect 4 | 4 split.

With pivot = $a[\text{end}] = 9$ (the maximum), the partition returns $p = 8$, a split of 8 | 0. Every level then removes only the pivot, which costs $(N-1)+(N-2)+\dots+1 = N(N-1)/2$ comparisons: $O(N^2)$, with recursion depth N .

(c) Sorted data, $N = 1000$:

- Pivot = last: **499,500** comparisons ($= 1000 \cdot 999/2$), recursion depth 1000, so $O(N^2)$.
- Median-of-3: **7,987** partition comparisons, recursion depth 10. This is below $N \log_2 N$, which is about 10,000, i.e. $O(N \log N)$. Picking the median costs at most 3 extra comparisons per partition.
- Reverse-sorted data: 499,500 vs 14,378.
- Random data changes little (one run: about 10,400 vs 9,000), because a random last element is already a reasonable guess.

It does **not** remove every worst case, because it is only a better guess of the median. For example, if all values are equal, Lomuto puts every element ($\leq$ pivot) on the left. So 1000 equal values still take 499,500 comparisons with median-of-3. The worst case stays $O(N^2)$; the average is $O(N \log N)$.

Verified: `./trace med3 <array>:`

```
first a[0]=8, middle a[3]=1, last a[6]=5 -> median (pivot) = 5
first a[0]=1, middle a[4]=5, last a[8]=9 -> median (pivot) = 5
first a[0]=9, middle a[4]=5, last a[8]=1 -> median (pivot) = 5
first a[0]=20, middle a[3]=17, last a[7]=6 -> median (pivot) = 17
first a[0]=12, middle a[2]=7, last a[5]=10 -> median (pivot) = 10
```

(b) `./v lomuto 1 2 3 4 9 6 7 8 5` gives `end swap(a[4],a[8]) => 1 2 3 4 5 6 7 8 9 return 4.` `./v lomuto 1 2 3 4 5 6 7 8 9` gives `return 8.`

(c) `./v qcount <kind> 1000`: my own quicks + partition1, with the median-of-3 swapped into $a[\text{end}]$; random data from `srand(218)`:

```
last sorted N=1000 partition comps=499500 maxdepth=1000
med3 sorted N=1000 partition comps=7987 maxdepth=10
last random N=1000 partition comps=10442 maxdepth=25
med3 random N=1000 partition comps=8962 maxdepth=15
last reverse N=1000 partition comps=499500 maxdepth=1000
med3 reverse N=1000 partition comps=14378 maxdepth=21
med3 equal N=1000 partition comps=499500 maxdepth=1000
```

The sorted, reverse and equal counts are deterministic and match the original bank exactly. The random counts depend on the seed, so they are given only as 'about'.

S11 · complexity

Counting operations and runtime analysis. (a) Exactly how many comparisons and swaps does the instructor's bubble sort make on 6 5 4 3 2 1? Show the passes and derive the general formula from the loops. (b) Fill in the number of comparisons for $N = 6$, both in reverse order and already sorted, for: bubble (no flag), bubble (early exit), selection (max), insertion, mergesort, quicksort (Lomuto, pivot = last). (c) The

instructor's sort program reports that bubble sort took 0.175 s for N = 10,000 random values. Estimate the time for N = 40,000. What ratio would you expect for quicksort?

Answer

(a) The passes are 5 4 3 2 1 6, 4 3 2 1 5 6, 3 2 1 4 5 6, 2 1 3 4 5 6, 1 2 3 4 5 6, and pass 6 makes 0 comparisons. Every comparison finds a pair out of order, so every comparison swaps: **15 comparisons and 15 swaps**.

From the loops: the outer i goes 0..N-1 and the inner j goes 0..N-2-i. So the inner body runs (N-1)+(N-2)+...+1+0 = N(N-1)/2 times, which is 15 for N = 6. The worst-case cost is (comp + swap) x N(N-1)/2 = (comp+swap)(N^2/2 - N/2), which is O(N^2).

(b)

algorithm	reverse 6 5 4 3 2 1	sorted 1 2 3 4 5 6
bubble (no flag)	15 comps, 15 swaps	15 comps, 0 swaps
bubble (early exit)	15 comps, 15 swaps	5 comps (1 pass)
selection (max)	15 comps, 3 swaps	15 comps, 0 swaps
insertion	15 comps, 15 shifts	5 comps, 0 shifts
mergesort	7 comps	9 comps
quicksort (Lomuto)	15 comps	15 comps

The formulas behind the table:

- $N(N-1)/2$ for bubble and selection (always), and for insertion and quicksort in the worst case.
- $N-1$ for insertion and flagged bubble on sorted data.
- At most about $N \log_2 N$ for mergesort.

(c) Bubble sort is $O(N^2)$. Multiplying N by 4 multiplies the work by $4^2 = 16$, so the estimate is $0.175 \times 16 =$ about **2.8 s**. The number of comparisons really grows exactly 16x: 49,995,000 becomes 799,980,000.

Measured CPU times do not follow the model exactly. On the test machine, repeated runs of the instructor's sort.cpp gave 0.17-0.26 s for N = 10,000 and 4.3-5.6 s for N = 40,000, ratios of about 18x to 32x from run to run. Timings are noisy and depend on the hardware and on memory behaviour, so on the exam give the model answer of 16x. What matters is that the growth is quadratic: linear growth would give only 4x.

Quicksort is $O(N \log_2 N)$. The expected ratio is $4 \times \log_2(40000)/\log_2(10000) = 4 \times 15.29/13.29 =$ about **4.6x**. Measured with sortp.cpp: 0.0011-0.0014 s became 0.0048-0.0067 s, about 4x to 6x.

Verified: ./v bubble 6 5 4 3 2 1 and ./trace bubble 6 5 4 3 2 1 give passes 5 4 3 2 1 6, 4 3 2 1 5 6, 3 2 1 4 5 6, 2 1 3 4 5 6, 1 2 3 4 5 6 and 15 comparisons / 15 swaps.

./v counts 6 5 4 3 2 1 prints bubble 15/15, bflag 15/15, selmax 15/3, ins 15/15, merge 7, quick 15.

./v counts 1 2 3 4 5 6 prints bubble 15/0, bflag 5/0, selmax 15/0, ins 5/0, merge 9, quick 15.

./counts random 40000 gives bubble comps=799980000; $10000 \times 9999/2 = 49,995,000$, and the ratio is 16.0.

Timings used the instructor's own programs: `g++ sort.cpp support.cpp -o sort`, `g++ sortp.cpp support.cpp -o sortp`, with data from `mkdata.cpp` (`srandomdev` replaced by `srandom(218)`): `./mkdata 10000 1 100000 > d10000.txt`, and the same for 40000.

- Bubble runs of `./sort 10000 < d10000.txt / ./sort 40000 < d40000.txt`: 0.211004/4.89185, 0.174757/5.64013, 0.176/4.43386 (25.2x), 0.226638/4.30269 (19.0x), 0.248938/4.41043 (17.7x).
- Quicksort runs of `./sortp 10000 < d10000.txt / ./sortp 40000 < d40000.txt`: 0.001399/0.006709, 0.001136/0.004793, 0.001215/0.004849, 0.001144/0.006531, 0.001328/0.006342, 0.001179/0.004839.
- Time per comparison for $N = 5000..40000$ ranged from 4.1 to 5.6 ns with no clear jump at the 48 KiB L1 size, so no single cause is claimed.

Python: $4 * \log_2(40000) / \log_2(10000) = 4.602$.

S12 · short-answer

Stability. What does it mean for a sorting algorithm to be *stable*?

The records 5a 3 5b 1 are sorted by the number only; the letter just shows the original order of the two 5s. Give the result of each algorithm and say whether it is stable: (a) bubble sort (instructor's version, swap if `a[j] > a[j+1]`) (b) insertion sort (shift while the sorted element is `>` the one being inserted) (c) selection sort (find max with the lecture's `if (a[j] > a[max_loc])`, swap it to the end) (d) mergesort with the lecture merge `if (A[i] < B[j])` (e) mergesort with `if (A[i] <= B[j])` (f) quicksort with `sortp.cpp`'s Lomuto partition1

Explain (c) and (d) step by step. Why can stability matter?

Answer

A sort is **stable** if records with equal keys keep their original relative order after sorting (here, 5a stays before 5b).

algorithm	result	stable?
(a) bubble	1 3 5a 5b	yes: it swaps only when <code>a[j] > a[j+1]</code> strictly, so equal neighbours are never swapped
(b) insertion	1 3 5a 5b	yes: it shifts only elements strictly greater than the one being inserted
(c) selection (max)	1 3 5b 5a	no
(d) mergesort, <code>A[i] < B[j]</code>	1 3 5b 5a	no
(e) mergesort, <code>A[i] <= B[j]</code>	1 3 5a 5b	yes
(f) quicksort (Lomuto)	1 3 5b 5a	no

(c) In pass 1, the max search uses `a[j] > a[max_loc]`, so it keeps the first 5 (5a, at index 0). 5a is swapped with the last element, 1, giving `1 3 5b 5a`. This long-distance swap jumped 5a over 5b, and the remaining passes do not change their order.

(d) The split is `5a 3 | 5b 1`, and the first merges give `3 5a` and `1 5b`. In the final merge:

- 3 vs 1: take 1.
- 3 vs 5b: take 3.
- 5a vs 5b: `5 < 5` is false, so the else branch takes **B's 5b first**, giving `1 3 5b 5a`.

Using `<=` takes from the left half on ties, which makes the sort stable.

(f) Quicksort swaps elements over long distances around the pivot, so it is not stable.

Why it matters: stability lets you sort records by a second key without destroying the order from the first key. For example, if students are already sorted by name and you then sort them stably by grade, students with the same grade stay in name order.

Verified: `stable.cpp` (my own implementations of the instructor's loops on records {key, tag}, comparing only the key; `g++ -std=c++11 -Wall -Wextra, ./stable 5a 3 5b 1`:

```
input      : 5a 3 5b 1
bubble     : 1 3 5a 5b
insertion  : 1 3 5a 5b
sel max    : 1 3 5b 5a
sel min    : 1 3 5b 5a
merge <    : 1 3 5b 5a
merge <=   : 1 3 5a 5b
quick Lom  : 1 3 5b 5a
```

Why the question must fix `>`: `stable_ge.cpp`, the same code with `>=` in the max search, prints `sel max : 1 3 5a 5b`.

Extra checks:

- `./stable 2a 2b` gives `merge < : 2b 2a` and `merge <= : 2a 2b`.
- `./stable 3a 1 3b 2 3c` gives `1 2 3a 3b 3c` for bubble and `merge(<=)`, and `1 2 3b 3c 3a` for selection max.

S13 · short-answer

Which algorithm for which situation? Choose bubble, selection, insertion, mergesort or quicksort, and justify your choice: (a) 10,000 values that are already sorted except for a few elements out of place. (b) 1,000,000 random integers, where you want the fastest average case with little extra memory. (c) Records where equal keys must keep their original order, and the time must be $O(N \log N)$ even in the worst case. (d) Very large records where every swap/write is expensive. (e) A tiny array (about 10 elements), or just checking whether data is already sorted. (f) Why is the instructor's quicksort (pivot = `a[end]`) a bad choice for (a)?

Answer

(a) Insertion sort. It makes about $(N-1) + (\text{number of inversions})$ comparisons, and nearly sorted data has few inversions, so it runs in almost $O(N)$. Bubble sort with early exit also finishes in a few passes if every

misplaced element is only a few places from its final position. But a small value that sits far too far right needs one pass per place.

(b) Quicksort. It is $O(N \log N)$ on average and sorts **in place**: it needs no work array, only the recursion stack ($O(\log N)$ levels on average). Mergesort actually makes somewhat fewer comparisons (about 120,000 vs 160,000 for 10,000 random values), but it needs an extra N -element work array and copies the data into it and back at every level. The 'little extra memory' requirement decides for quicksort, which was also faster in practice here (1,000,000 random ints: about 0.16 s vs 0.21 s). Use median-of-3 to avoid the sorted-input worst case.

(c) Mergesort (with the merge using `<=`). It is stable and $O(N \log N)$ in the best, average and worst case, because the split is always in half. The price is an $O(N)$ work array.

(d) Selection sort. It makes at most $N-1$ swaps, and each swap puts one element directly into its final place. It still makes $N(N-1)/2$ comparisons, though.

(e) For a tiny array: **insertion sort**, which is simple, has no recursion overhead and does little work on small N . g++'s own `std::sort` also stops partitioning once a piece has 16 or fewer elements and finishes with insertion sort. To check whether data is already sorted: **bubble sort with early exit**, which takes one pass of $N-1$ comparisons.

(f) On sorted or nearly sorted data, the last element is (almost) the maximum. Nearly every partition then splits $N-1 \mid 0$, costing about $N^2/2$ comparisons: $O(N^2)$, with recursion depth close to N and a risk of stack overflow.

Measured results (the instructor's loops, $N = 10,000$). The nearly sorted data was 1..10000 with 101 random, non-overlapping adjacent pairs swapped; exact values for the nearly sorted and random data depend on the data.

Nearly sorted:

- insertion: 10,100 comparisons (= 9,999 + 101 inversions)
- bubble with early exit: 19,997 (2 passes)
- mergesort: about 69,000
- selection: 49,995,000 (always $N(N-1)/2$)
- quicksort (pivot = last): about 49.5 million, recursion depth about 9,900

Random:

- mergesort: about 120,000 comparisons
- quicksort: about 160,000 (recursion depth 35)
- insertion: about 25 million
- selection makes only about 10,000 swaps, vs about 25 million swaps/shifts for bubble and insertion.

Verified: counts.cpp (my own implementations of the instructor's loops; g++ -std=c++11 -O2 -Wall -Wextra; srand(218)).

```
./counts nearly 10000:
```

```
bubble      comps=49995000 swaps=101
bubble(flag) comps=19997 swaps=101
selection(max) comps=49995000 swaps=101
```

```

insertion      comps=10100 shifts=101
mergesort      comps=69051 element assignments=267232 sorted=1
quick(Lomuto)  comps=49519112 swaps=49528909 element assignments=148586727 max depth=9899

```

./counts random 10000:

```

bubble        comps=49995000 swaps=24931314
selection(max) comps=49995000 swaps=9992
insertion      comps=24941305 shifts=24931314
mergesort      comps=120456 element assignments=267232 sorted=1
quick(Lomuto)  comps=161633 swaps=85656 element assignments=256968 max depth=35

```

The element assignments (267,232 vs 256,968) show that the original 'mergesort moves data more (133,616 vs 87,393)' compared copies with swaps.

qm_time.cpp (instructor-style quicks+partition1 vs mergesort with a work array, built without optimization with support.cpp, 1,000,000 values from randomInRange): three runs gave quicksort 0.163524 s, mergesort 0.207445 s, 0.167032 / 0.215771, 0.158097 / 0.204505.

In libstdc++, /usr/include/c++/13/bits/stl_algo.h:1848 has enum { _S_threshold = 16 };, used by __introsort_loop (line 1922, while (__last - __first > int(_S_threshold))) and then __final_insertion_sort (line 1950).

S14 · find-the-bug

Generic bubble sort in C style (void *), from sortb.cpp:

```

int comp_int(const void *a, const void *b) {
    int *a1 = (int *)a; // cast to int *
    int *b1 = (int *)b;
    if (*a1<*b1) return -1;
    else if (*a1>*b1) return 1;
    else return 0;
}

void bubbleSort(void *arr, const int num, const int esize, int (*comp)(const void *, const void *))
{
    void *a, *b;
    void *temp = malloc(esize);
    for(int i=0;i<num;i++) { // outer loop
        for(int j=0;j<num-1-i;j++) {
            a = (char *)arr + (j*esize);
            b = (char *)a + esize;
            if (comp((const void *)a,(const void *)b)>0) {
                memcpy(temp,a,esize);
                memcpy(a,b,esize);
                memcpy(b,temp,esize);
            }
        }
    }
    free(temp);
}

```

(a) What is a void *? Why is the address of element j computed as (char *)arr + (j*esize) and not arr + j? If an int array starts at address 0x1000, what address does the code compute for j = 3? (b) Explain malloc(esize), the three memcpy calls and free(temp). What happens if free(temp) is left out? (c) What

must a compare function return? Write `comp_int_desc` (descending order) and `comp_double`. (d) Write the call that sorts `double d[5]` with this `bubbleSort`, and the call that sorts `int *data` (num values) with the C library `qsort`. (e) Each of the following is wrong. Find the bug and say what goes wrong. For i), also give the printed order of `d = {2.5, 1.25, 2.75, 1.5, 2.0}` after the sort. i) `int comp_double_bad(const void *a, const void *b) { return (int)(*(const double *)a - *(const double *)b); }` ii) `bubbleSort((void *)d, 5, sizeof(int), comp_double);`

Answer

(a) A `void *` is a raw address with no type. The compiler does not know the size of what it points to, so:

- It cannot be dereferenced: `*p` gives the error `'void*' is not a pointer-to-object type`.
- Standard C++ does not allow arithmetic on it. clang reports `error: arithmetic on a pointer to void`. g++ accepts `arr + j` only as a GNU extension and warns `pointer of type 'void *' used in arithmetic` (even without `-Wall`; `-pedantic-errors` makes it an error).
- Even under that extension, `arr + j` moves only **j bytes**, not j elements, so it would point into the middle of an element.

A `char` is exactly 1 byte, so `(char *)arr + j*esize` moves exactly *j* **bytes**. That is the start of element *j* for any element type, and `b = (char *)a + esize` is the next element. For ints (`esize = 4`): `0x1000 + 34 = 0x100C`.

(b) The element type is unknown, so the temporary cannot be declared as `int temp`. Instead, `malloc(esize)` allocates `esize` bytes on the heap for it.

`memcpy(dest, src, n)` copies `n` bytes. The three calls `copy temp <- a`, `a <- b`, `b <- temp`, which swaps two elements of any size.

`free(temp)` gives the block back. Without it, every call leaks `esize` bytes: a **memory leak**. For an int sort, `valgrind` reports 4 bytes in 1 blocks are definitely lost.

(c) It must return a negative value (-1) if `a < b`, 0 if `a == b`, and a positive value (1) if `a > b`. `bubbleSort` swaps when `comp(a,b) > 0`, i.e. when `a` must come after `b`.

```
// descending: reverse the answer of comp_int
int comp_int_desc(const void *a, const void *b) {
    return comp_int(b, a);
}

// compare for double
int comp_double(const void *a, const void *b) {
    const double *a1 = (const double *)a;
    const double *b1 = (const double *)b;
    if (*a1 < *b1) return -1;
    else if (*a1 > *b1) return 1;
    else return 0;
}
```

(d)

```
bubbleSort((void *)d, 5, sizeof(double), comp_double);
qsort((void *)data, num, sizeof(int), comp_int); // #include <stdlib>
```

`qsort` takes the same 4 pieces of information: the base address, the number of elements, the size of one element and the compare function.

(e)

i) The difference is truncated to an int, so every difference with absolute value below 1 becomes 0: $2.0 - 2.75 = -0.75$ becomes 0, and $2.75 - 2.0 = 0.75$ becomes 0. Values closer than 1 therefore look equal and are never swapped. Tracing the bubble sort gives 1.25 1.5 2.5 2.75 2, which is wrong. Use the `</>` version from (c) instead.

ii) `esize` is 4, but a double is 8 bytes. The function treats the first 20 bytes as 5 elements of 4 bytes each: it compares 8-byte values starting at 4-byte steps and swaps half-doubles. The array is scrambled: some values become meaningless bit patterns, and the rest is not sorted. On the test machine it printed 5.3011e-315 5.30628e-315 2.75 1.5 2. With the correct `sizeof(double)` it prints 1.25 1.5 2 2.5 2.75.

The compiler cannot catch either mistake, because everything is a `void *`. That is the weakness of the C approach, and C++ templates fix it.

Verified: `generic.cpp` contains the `sortb.cpp` bubbleSort and `comp_int` copied unchanged, plus the answer's `comp_int_desc` and `comp_double` (`g++ -std=c++11 -Wall -Wextra`, no warnings). `./generic`:

```
&di[0]=0x7ffde6bb0500 (char*)di+3*sizeof(int)=0x7ffde6bb050c &di[3]=0x7ffde6bb050c
int asc : 10 20 30 40 50 60
int desc: 60 50 40 30 20 10
double ok      : 1.25 1.5 2 2.5 2.75
double bad cmp : 1.25 1.5 2.5 2.75 2
esize=sizeof(int): 5.3011e-315 5.30628e-315 2.75 1.5 2
qsort  : 10 20 30 40 50 60
```

`valgrind --leak-check=full`: in use at exit: 0 bytes, ERROR SUMMARY: 0 errors. With `-DLEAK` (`free(temp)` removed), `valgrind` reports 4 bytes in 1 blocks are definitely lost (int sorts) and definitely lost: 28 bytes in 5 blocks.

Compiler diagnostics:

- `voidarith.cpp` (`void *a = arr + j * esize;`): plain `g++ -std=c++11 -c` gives warning: pointer of type 'void *' used in arithmetic [-Wpointer-arith]; with `-pedantic-errors` it becomes error: ...; `clang++` gives error: arithmetic on a pointer to void.
- `voidderef.cpp` (`return *p;`): `g++` gives error: 'void*' is not a pointer-to-object type.
- `voidstep.cpp`: `arr+3` moves 3 bytes; `(char*)arr+3*4` moves 12 bytes -> value 40.

The instructor's `sortb.cpp` and `sortq.cpp`, built with `support.cpp` and run on 40 10 50 20 60 30, both print 10 20 30 40 50 60. `/usr/include/stdlib.h:970` has extern `void qsort (void *__base, size_t __nmem, size_t __size, ...`

S15 · find-the-bug

Template bubble sort, from `sortt.cpp`:

```
int comp_int(const int &a, const int &b) {
    return (a-b);
}

void swap(int &a, int &b) {
    int temp = a;
```

```

    a = b;
    b = temp;
}

template <class T>
void bubbleSort(T *arr, const int num, int (*comp)(const T&, const T&)) {
    for(int i=0;i<num;i++) { // outer loop
        for(int j=0;j<num-1-i;j++) {
            if (comp(arr[j],arr[j+1])>0) {
                swap(arr[j],arr[j+1]);
            }
        }
    }
}
// in main: bubbleSort<int>(data,num,comp_int);

```

(a) What does the compiler do with the template and the call `bubbleSort<int>(data,num,comp_int)`? (b) The program is built with the course's compile line `g++ sortt.cpp support.cpp -o sortt` (no optimization). What does it print (sorted part) for the input `2147483647 -2147483648 5` (`./sortt 3 < ovf.txt`)? Explain, and fix `comp_int`. (c) What happens if main calls `bubbleSort<double>(dd,3,comp_double)`, with a correct `comp_double`? Fix it. (d) A student does the following:

- puts only the declaration `template <class T> void bubbleT(T *arr, const int num);` in `bubblet.h`,
- puts the template body in `bubblet.cpp`,
- calls `bubbleT<int>(arri,5)` in `main.cpp`,
- builds with `g++ main.cpp bubblet.cpp`.

What happens, and why? Where must template code go? (e) What must a type `T` provide to be used with this template?

Answer

(a) A template is a code pattern with a placeholder data type `T`; by itself it produces no code. When the compiler sees `bubbleSort<int>(...)`, it **generates a specific function** with `T` replaced by `int`: `void bubbleSort<int>(int*, int, int (*)(const int&, const int&))`. Every other type that is used (`bubbleSort`, `bubbleSort<std::string>`) gets its own generated copy. So one source gives many type-checked functions, unlike `void *`.

(b) It prints `5 2147483647 -2147483648`, which is **wrong**, because `a-b` overflows `int`. With the unoptimized build the result wraps around (two's complement):

- Pass 1, `j=0`: `comp(2147483647, -2147483648) = 4294967295`, which does not fit in an `int` and wraps to `-1`. That means '`a < b`', so no swap.
- Pass 1, `j=1`: `comp(-2147483648, 5) = -2147483653` wraps to `2147483643 > 0`, so swap, giving `2147483647 5 -2147483648`.
- Pass 2: `comp(2147483647, 5) = 2147483642 > 0`, so swap, giving `5 2147483647 -2147483648`.

Signed overflow is **undefined behaviour**; the sanitizer reports `signed integer overflow: 2147483647 - -2147483648 cannot be represented in type 'int'`. Undefined means the result cannot be relied on. The same source built with `-O2` happened to print the correct order `-2147483648 5 2147483647`. The fix is to compare instead of subtract:


```
int comp_int(const int &a, const int &b) {
    if (a < b) return -1;
    else if (a > b) return 1;
    else return 0;
}
```

Now it prints `-2147483648 5 2147483647` with any compile options.

(c) Generating bubbleSort fails to compile: cannot bind non-const lvalue reference of type 'int&' to a value of type 'double' at `swap(arr[j],arr[j+1])`, because the only swap takes `int&`. Fix: make swap a template too (the lecture calls it `swapT`):

```
template <class T>
void swapT(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}
```

Then call `swapT(arr[j],arr[j+1])` in bubbleSort. A template `compT` that uses `<` and `>` also works for any `T`, e.g. `compT<double>` or `compT<std::string>`.

(d) The build fails with a link error: undefined reference to 'void bubbleT<int>(int*, int)'.

`bubblet.cpp` is compiled on its own, and nothing in it uses `bubbleT`, so no code is generated: `bubblet.o` contains no symbols at all. `main.cpp` only sees the declaration, so the compiler assumes the function is defined elsewhere and leaves it to the linker, which cannot find it.

The compiler needs the **template source code** at the point where it is used. So put the **whole template (declaration + body) in the .h header** and `#include` it.

(e) `T` must support every operation that the template, and the functions it calls, apply to `T` values:

- In `sortt.cpp` as written, the template body itself only passes `T` values to `comp` and `swap`. So there must be a compare function `int comp(const T&, const T&)` and a swap that accepts `T`; the `int`-only swap is exactly what breaks in (c).
- With the generic `compT/swapT`, `T` needs `operator<` and `operator>` (used by `compT`), plus copy construction and assignment (`T temp = a; a = b;` in `swapT`).
- It needs `<<` if the data is printed.

For example, `struct Point {int x, y;}` without these operators gives no match for 'operator<' (operand types are 'const Point' and 'const Point') and the same for `operator>`.

Verified: Instructor `sortt.cpp`: `g++ -std=c++11 -Wall -Wextra -c sortt.cpp; nm -C sortt.o` lists `W void bubbleSort<int>(int*, int, int (*)(int const&, int const&)), T swap(int&, int&)` and `T comp_int(int const&, int const&)`.

Course build (`g++ -std=c++11 -Wall -Wextra sortt.cpp support.cpp -o sortt,no -O`): `./sortt 3 < ovf.txt` prints the input block, then the sorted block `5, 2147483647, -2147483648`. `./sortt 5 < small.txt (30 -7 12 0 5)` prints `-7 0 5 12 30`.

`g++ -fsanitize=undefined` gives `sortt.cpp:27:14: runtime error: signed integer overflow: 2147483647 - -2147483648 cannot be represented in type 'int'`. `g++ -O2 sortt.cpp support.cpp -o`

sortt_02; ./sortt_02 3 < ovf.txt prints the sorted block -2147483648, 5, 2147483647, which is the undefined-behaviour point.

(c) sortt_double.cpp (sortt.cpp plus comp_double and a bubbleSort<double>(dd, 3, comp_double) call) fails with error: cannot bind non-const lvalue reference of type 'int&' to a value of type 'double' and note: initializing argument 1 of 'void swap(int&, int&)'.
| wc -l gives 0.

(d) In tsplit/, g++ main.cpp bubblet.cpp fails with main.cpp:(.text+0x4b): undefined reference to 'void bubbleT<int>(int*, int)' and collect2: error: ld returned 1 exit status. nm -C bubblet.o

The header-only fix, tfixed/bubblet.h (swapT, compT, bubbleSort), prints -2147483648 5 2147483647, 1.25 1.5 2 2.5 2.75 and apple banana fig pear.

(e) tfixed/noop.cpp gives bubblet.h:13:9: error: no match for 'operator<' (operand types are 'const Point' and 'const Point') and bubblet.h:14:14: error: no match for 'operator>' (operand types are 'const Point' and 'const Point').

Compiling, memory, pointers and references

CMP-1 · output-prediction

The instructor's `hello.cpp` (8/18) is compiled with `g++ hello.cpp -o hello`:

```
#include <iostream>
#include <string>

int main(int argc, char *argv[]) {
    std::cout << argv[0] << std::endl;
    std::cout << "Hello World\n";
    for(int i=0;i<argc;i++)
        std::cout << "index: " << i << " value=" << argv[i] << std::endl;
    return 0;
}
```

(a) What is printed by `./hello red "green blue" 42`? What is the value of `argc`? (b) What is printed by `./hello` with no arguments? (c) What appears on the screen for `./hello x y > out.txt`, and what does `out.txt` contain? (d) What is the type of `argv[3]` in (a)? How do you get the number 42 as an `int`?

Answer

(a) `argc = 4`. The program name counts as `argv[0]`. The shell removes the quotes, and they make `green blue` a single argument.

```
./hello
Hello World
index: 0 value=./hello
index: 1 value=red
index: 2 value=green blue
index: 3 value=42
```

(b) `argc = 1`, so the only argument is the program name:

```
./hello
Hello World
index: 0 value=./hello
```

(c) Nothing appears on the screen. `>` sends standard output (`std::cout`) to the file, so `out.txt` holds:

```
./hello
Hello World
index: 0 value=./hello
index: 1 value=x
index: 2 value=y
```

The shell handles `> out.txt` itself, so it is **not** passed to the program, and `argc` is 3.

(d) `argv[3]` is a `char *`: a C string holding the characters `'4' '2' '\0'`, not a number. Convert it with `int n = std::stoi(argv[3]);` (needs `<string>`), as `sort.cpp` and `mkdata.cpp` do with `std::stoi(argv[1], &sz)`.

- `std::stoi` throws `std::invalid_argument` when the text does not **start** with a number, e.g. `five` (see CMP-4).
- Text that only starts with digits, such as `5abc`, is silently converted to 5.

Verified: In `scratchpad/verify_bank_memory/`, `g++ -std=c++11 -Wall -Wextra hello.cpp -o hello` compiled with no warnings. `./hello red "green blue" 42` printed `./hello, Hello World, index: 0 value=./hello, index: 1 value=red, index: 2 value=green blue, index: 3 value=42. ./hello` printed `./hello, Hello World, index: 0 value=./hello. ./hello x y > out.txt` printed nothing on screen, and `cat out.txt` showed the 5 lines ending `index: 2 value=y. ./hello a < data.txt` listed only index 0 and index 1, so the redirection is not in `argv`. `./sort 5abc < data.txt` ran with `n=5` (number: 5 took `2e-06`). `./sort five` gave `terminate called after throwing an instance of 'std::invalid_argument' / what(): stoi`.

CMP-2 · short-answer

The instructor's `sort.cpp` includes `hw.h` and `support.h` and calls `getCPUtime()`, which is **defined** in `support.cpp`. The correct build is `g++ sort.cpp support.cpp -o sort`. For each case, name the stage that reports the problem (**preprocessor, compiler, assembler, linker, or run time**) and describe the message:

(a) `g++ sort.cpp -o sort`, with `support.cpp` left off the command line (b) `#include "hw.h"` mistyped as `#include "hw2.h"` (c) the `;` is missing in `int temp = a` inside `swap()` (d) the call is written `bubblesort(data,num);` (lower-case s) (e) `printData` is only declared, as `void printData(int *d, const int n);`, and its body was deleted (f) the instructor's `mkdata.cpp`, which calls `srandomdev()`, is compiled on Linux (g) the program is run as `./sort` with no number

Also: which `g++` option stops after each stage, and why does `g++ -c sort.cpp` succeed in case (a)?

Answer

case	stage	message (g++ 13 / Linux)
(a) support.cpp omitted	linker (ld)	undefined reference to 'getCPUtime()' ... collect2: error: ld returned 1 exit status
(b) hw2.h	preprocessor	fatal error: hw2.h: No such file or directory / compilation terminated.
(c) missing ;	compiler (syntax)	error: expected ',' or ';' before 'a', reported at the next line (a = b;)
(d) bubblesort	compiler (the name is not declared)	error: 'bubblesort' was not declared in this scope; did you mean 'bubbleSort'?
(e) declared but not defined	linker	undefined reference to 'printData(int*, int)'. The declaration is enough for the compiler; the linker cannot find the body. The top-level const of const int n is not part of the signature, so the message shows int.
(f) srandomdev on Linux	compiler	error: 'srandomdev' was not declared in this scope. The function exists only on macOS/BSD, where the instructor compiles. Fix: srandom(time(NULL)); with #include <ctime>.
(g) ./sort	run time. This is not a build error; the program checks argc!=2 itself.	Usage: ./sort <number of values> on stderr, then return -1 (the shell shows exit status 255).

Stages and options:

- `g++ -E sort.cpp` stops after **preprocessing**. Every `#include` d header is pasted in, macros are expanded and all `#` lines are gone (hw.h's `#define DPRINT` no longer appears). The output is tens of thousands of lines.
- `g++ -S sort.cpp` stops after **compiling to assembly** (`sort.s`).
- `g++ -c sort.cpp` stops after **assembling** into an object file (`sort.o` , relocatable machine code).
- `g++ sort.o support.o -o sort` **links** the object files and the C++ library into the executable.

Why `g++ -c sort.cpp` works in (a):

- The compiler only needs the **declaration** `double getCPUtime(void);` from support.h.
- `sort.o` just records an *unresolved* symbol (`nm` shows `U getCPUtime()`).
- The definition (`T getCPUtime()`) is in support.o, and only the **linker** needs it.

So support.cpp must be on the compile/link line; including its header is not enough.

Verified: Run in scratchpad/verify_bank_memory/stages/ with `g++ -std=c++11 -Wall -Wextra:`

- (a) `/usr/bin/ld: ... sort.cpp:(.text+0x323): undefined reference to 'getCPUtime()' and collect2: error: ld returned 1 exit status.`
- (b) `b.cpp:4:10: fatal error: hw2.h: No such file or directory and compilation terminated.`
- (c) `c.cpp:25:3: error: expected ',' or ';' before 'a'; line 25 is a = b;.`
- (d) `d.cpp:79:5: error: 'bubblesort' was not declared in this scope; did you mean 'bubbleSort'?`

- (e) e.cpp:(.text+0x2a0): undefined reference to 'printData(int*, int)', while g++ -c e.cpp compiled fine.
- (f) mkdata.cpp:16:5: error: 'srandomdev' was not declared in this scope; did you mean 'srandom_r'? The srandom(time(NULL)) version compiled cleanly with -pedantic.
- (g) ./sort printed Usage: ./sort <number of values>, exit=255.

Stages: g++ -std=c++11 -E sort.cpp gave 31689 lines (37625 with the default -std) and 0 occurrences of DPRINT. -S produced sort.s. -c produced sort.o, which file reports as ELF 64-bit LSB relocatable. nm -C sort.o shows U getCPUtime() and nm -C support.o shows T getCPUtime(). g++ sort.o -o sortX failed with ld returned 1 exit status, and g++ sort.o support.o -o sortOK linked.

CMP-3 · output-prediction

The instructor's hw.h defines:

```
#ifdef DEBUG
#define PDEBUG 1
#else
#define PDEBUG 0
#endif
#define DPRINT(fmt, ...) do {\
    if (PDEBUG) {\
        char str[100];\
        snprintf(str, 100, fmt, ##__VA_ARGS__);\
        std::cerr << "DEBUG-->";\
        std::cerr << __FILE__ << ":" << __LINE__ << ":\n";\
        std::cerr << __func__ << "():\n" << str;\
    }\
} while (0)
```

dp.cpp:

```
#include <iostream>
#include "hw.h"

int main() {
    int sum = 0;
    for (int i = 1; i <= 3; i++) {
        sum += i;
        DPRINT("i=%d sum=%d\n", i, sum); // this is line 8
    }
    std::cout << "sum=" << sum << std::endl;
    return 0;
}
```

(a) What does g++ dp.cpp -o dp; ./dp print? (b) What does g++ -DDEBUG dp.cpp -o dpd; ./dpd print? (c) What appears on the screen for ./dpd 2> /dev/null? For ./dpd > out.txt, what appears on the screen and what is in out.txt? (d) Which compilation stage turns DPRINT on or off? Why is the macro wrapped in do { ... } while (0)?

Answer

(a) Without -DDEBUG, PDEBUG is 0, so every DPRINT becomes if (0) { ... } and prints nothing:

```
sum=6
```

(b) `-DDEBUG` defines the symbol `DEBUG`, which makes `PDEBUG` 1. Each call prints to **stderr** the file, the line of the `DPRINT` call (`__LINE__` is expanded where the macro is used), the function name and the message:

```
DEBUG-->dp.cpp:8:main():i=1 sum=1
DEBUG-->dp.cpp:8:main():i=2 sum=3
DEBUG-->dp.cpp:8:main():i=3 sum=6
sum=6
```

(c) `DPRINT` writes to `std::cerr` (stream 2) and the result goes to `std::cout` (stream 1).

- `./dpd 2> /dev/null` throws away `stderr`, so the screen shows only `sum=6`.
- `./dpd > out.txt` shows the three `DEBUG-->` lines on the screen, and `out.txt` contains only `sum=6`.

So debug output never gets mixed into data that is redirected to a file.

(d) The **preprocessor** does this.

- `#ifdef DEBUG` is decided before compiling and sets `PDEBUG` to 1 or 0.
- `g++ -E` shows `if (0)` without the flag and `if (1)` with `-DDEBUG`.
- The compiler never executes, and optimizes away, the `if (0)` block.
- The source does not change; only the compile line does.

`do { ... } while (0)` makes the macro behave like **one statement** that needs the `;` after it, so `if (x) DPRINT("..."); else ...` works. With a plain `{ ... }` macro, the `;` after the braces ends the `if`, and the compiler reports `error: 'else' without a previous 'if'`.

Verified: Run in `scratchpad/verify_bank_memory/dprint/` with the instructor's hw.h. `./dp` printed `sum=6`. `./dpd` printed `DEBUG-->dp.cpp:8:main():i=1 sum=1, DEBUG-->dp.cpp:8:main():i=2 sum=3, DEBUG-->dp.cpp:8:main():i=3 sum=6` and `sum=6`. `./dpd 2>/dev/null` printed only `sum=6`. `./dpd > out.txt` showed the 3 `DEBUG` lines on the screen, and `cat out.txt` gave `sum=6`. `-Wall -Wextra` gave no warnings with or without `-DDEBUG`. `g++ -E dp.cpp | grep -o 'if (0)'` matched, and `g++ -E -DDEBUG dp.cpp` showed `do { if (1) { char str[100]; sprintf(str, 100, "i=%d sum=%d\n", i, sum); ... << "dp.cpp" << ":" << 8 << ...`. In `dw.cpp`, `DPRINT` inside `if/else` compiled and printed `DEBUG-->dw.cpp:14:main():one arg`. The braces-only macro (`-DBRACES`) gave `dw.cpp:15:5: error: 'else' without a previous 'if'`.

CMP-4 · output-prediction

`data.txt`, made with `./mkdata 5 1 100 > data.txt`, contains one value per line: `42 7 19 3 25`. The instructor's `sort.cpp` (`loadData`, `bubbleSort`, `printData`, `getCPUtime`) is built with `g++ sort.cpp support.cpp -o sort`. (a) What does `./sort 5 < data.txt` print? (b) What does `./sort 3 < data.txt` print? (c) What does `./sort 8 < data.txt` print? Explain the extra values. (d) What happens with `./sort` and with `./sort five < data.txt`? (e) What does `< data.txt` do, and what happens with `./sort 5` without it?

Answer

(a) The program prints the loaded data, then the sorted data, then the time:

```

+++++
42
7
19
3
25
+++++
3
7
19
25
42
+++++
number: 5 took 2e-06

```

The time is the CPU seconds (user + system) measured by `getCPUtime()`. It is a tiny number, a few microseconds, and it changes from run to run.

(b) Only the first 3 values are read (42 7 19). The output is 42 7 19, then 7 19 42, then `number: 3 took ...`. The rest of the file is never read.

(c) `new int[8]` is allocated, but only 5 values exist.

- At end-of-file `std::cin >> temp[i]` fails and leaves the variable **unchanged**, so the last 3 elements are **never written**. They hold whatever was in that heap memory (uninitialized).
- On this run they happened to print as 0 0 0 (fresh memory from the OS is often zero), so the sorted output was 0 0 0 3 7 19 25 42. That is luck, not guaranteed.
- `valgrind` reports Conditional jump or move depends on uninitialised value(s).
- The program never checks `std::cin` for failure, e.g. with `if (!(std::cin >> temp[i]))`.

(d)

- `./sort` gives `argc != 2`, so it prints `Usage: ./sort <number of values>` to `stderr` and returns -1 (exit status 255).
- `./sort five` makes `std::stoi("five")` throw, and the program aborts:

```

terminate called after throwing an instance of 'std::invalid_argument'
what():  stoi

```

(e) `<` is **input redirection**. The shell connects the file to the program's standard input, so `std::cin` reads from `data.txt` instead of the keyboard.

- The program does not know or care, and `< data.txt` does not appear in `argv`.
- Without it, `./sort 5` waits for 5 numbers typed at the keyboard.
- Output redirection (`> sorted.txt`) and pipes (`./mkdata 5 1 100 | ./sort 5`) work the same way.

Verified: Built with `g++ -std=c++11 -Wall -Wextra sort.cpp support.cpp -o sort` (no warnings) in `scratchpad/verify_bank_memory/`, using `printf "42\n7\n19\n3\n25\n" > data.txt`.

- (a) printed exactly the listing, ending `number: 5 took 2e-06`.
- (b) printed 42 7 19 / 7 19 42 / `number: 3 took 2e-06`.

- (c) printed 42 7 19 3 25 0 0 0, then 0 0 0 3 7 19 25 42. valgrind ./sort_g 8 < data.txt reported 6x Conditional jump or move depends on uninitialised value(s) and Use of uninitialised value of size 8.
- eoftest.cpp (array pre-filled with 777, read 8 values from the 5-line file) printed 42 7 19 3 25 777 777 777 with fail=1 eof=1, so the elements at EOF stay unwritten.
- (d) ./sort printed Usage: ./sort <number of values>, exit=255. ./sort five < data.txt printed terminate called after throwing an instance of 'std::invalid_argument' / what(): stoi, exit=134.
- (e) ./mkdata 5 1 100 | ./sort 5 worked, using mkdata built with srand(time(NULL)).

PTR-5 · output-prediction

(a) What does each line print? (int is 4 bytes and a pointer is 8 bytes, 64-bit machine.) For L15, describe the addresses rather than giving exact values.

```
#include <iostream>
using namespace std;

int main() {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;

    cout << "L1: " << *p << endl;
    cout << "L2: " << *(p + 2) << endl;
    cout << "L3: " << *p + 2 << endl;
    cout << "L4: " << p[3] << endl;
    cout << "L5: " << *(a + 4) << endl;
    p++;
    cout << "L6: " << *p << endl;
    cout << "L7: " << *p++ << endl;
    cout << "L8: " << *p << endl;
    cout << "L9: " << (*p)++ << endl;
    cout << "L10: " << a[2] << endl;
    cout << "L11: " << p - a << endl;
    cout << "L12: " << &a[4] - &a[1] << endl;
    cout << "L13: " << sizeof(a) << " " << sizeof(p) << " "
        << sizeof(a) / sizeof(a[0]) << endl;
    cout << "L14: " << (a == &a[0]) << endl;
    cout << "L15: " << a << " " << &a << " " << &a[0] << endl;
    cout << "L16: " << (char *)(&a + 1) - (char *)a << " "
        << (char *)(&a + 1) - (char *)&a << endl;
    return 0;
}
```

(b) Which of these compile? `int *q = &a; / a++; / int (*pa)[5] = &a;` (c) In the instructor's generic `sortb.cpp`, element `j` of a `void *arr` is found with `(char *)arr + j*esize`. With `int data[10] = {0,10,...,90}`, `j = 2` and `esize = 4`, what do `*(int*)((char*)arr + j*esize)` and `*(int*)((int*)arr + j*esize)` give?

Answer

(a)

line	output	why
L1	10	p points to a[0]
L2	30	*(p+2) is a[2] . p+2 moves 2 elements , which is 8 bytes.
L3	12	* binds tighter than +: (*p)+2 = 10+2
L4	40	p[3] is the same as *(p+3)
L5	50	the array name decays to &a[0]; *(a+4) is a[4]
L6	20	after p++, p points to a[1]
L7	20	*p++ is *(p++) : it uses the old p (a[1]), then p moves to a[2]
L8	30	p points to a[2]
L9	30	(*p)++ prints 30, then a[2] becomes 31
L10	31	
L11	2	pointer difference counts elements , not bytes
L12	3	4 – 1
L13	20 8 5	sizeof the array = 5×4 = 20; sizeof a pointer = 8; 20/4 = 5 elements
L14	1	a decays to &a[0] (true prints 1)
L15	the same address printed three times (a stack address like 0x7ffc...)	a, &a and &a[0] all start at the same place
L16	4 20	a+1 moves one int (4 bytes); &a+1 moves one whole int[5] (20 bytes)

The addresses in L15 are equal, but the **types differ**. a (after decay) and &a[0] are `int*`, while &a is `int (*)[5]`, a pointer to the whole array. That is why the steps in L16 are different.

(b)

- `int *q = &a;` does **not** compile: error: cannot convert '`int (*)[5]`' to '`int*`' in initialization.
- `a++;` does **not** compile: error: lvalue required as increment operand. The array name is not a variable that can be changed; only a pointer like p can move.
- `int (*pa)[5] = &a;` compiles, and `(*pa)[1]` is 20.

(c)

- `(char*)arr + 2*4` moves 8 **bytes**, which is element 2, so the result is **20** (correct).
- `(int*)arr + 2*4` moves 8 **ints** (32 bytes), which is element 8, so the result is **80** (wrong element).

This is why the generic C code casts to `char*`: a char is 1 byte, so `j*esize` counts bytes.

Standard C++ does not allow arithmetic directly on a `void*`:

- clang++, the compiler on the instructor's Mac, rejects `arr + j*esize` with error: arithmetic on a pointer to void.
- g++ accepts it only as a GNU extension (it treats void as 1 byte) and warns, even without `-Wall`: pointer of type 'void *' used in arithmetic `[-Wpointer-arith]`.

Verified: scratchpad/verify_bank_memory/ptr/parith.cpp (exact code) compiled with `g++ -std=c++11 -Wall -Wextra` with no warnings. It printed L1: 10, L2: 30, L3: 12, L4: 40, L5: 50, L6: 20, L7: 20, L8: 30, L9: 30, L10: 31, L11: 2, L12: 3, L13: 20 8 5, L14: 1, L15: 0x7ffc74a550a0 0x7ffc74a550a0 0x7ffc74a550a0, L16: 4 20. valgrind -q was clean, and the clang++ build gave the same values. perr.cpp: -DE1 gave perr.cpp:5:14: error: cannot convert 'int (*)[5]' to 'int*' in initialization, -DE2 gave perr.cpp:9:5: error: lvalue required as increment operand, and the `int (*pa)[5] = &a;` version printed 20. voidp.cpp printed 20 80. With `-DVA (arr + j*esize)`, g++ (both with and without `-Wall`) gave warning: pointer of type 'void *' used in arithmetic `[-Wpointer-arith]` and printed 20. clang++ gave error: arithmetic on a pointer to void.

PTR-6 · find-the-bug

The programmer wants `printData` to print every element of any int array:

```
#include <iostream>
using namespace std;

// print all values of the array
void printData(int arr[]) {
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "n=" << n << ": ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int data[6] = {4, 8, 15, 16, 23, 42};
    cout << "in main: " << sizeof(data) / sizeof(data[0]) << endl;
    printData(data);
    return 0;
}
```

(a) What does it print (64-bit machine)? (b) What is the bug? (c) Fix it the way the instructor's sort programs do.

Answer

(a)

```
in main: 6
n=2: 4 8
```

(b) An array passed to a function **decays to a pointer to its first element**. `int arr[]` in a parameter list really means `int *arr`.

- Inside `printData`, `sizeof(arr)` is the size of a **pointer** (8 bytes), not of the array (24 bytes). So $n = 8/4 = 2$, and only 2 values are printed.
- In `main`, `data` is the real array, so `sizeof(data)/sizeof(data[0]) = 24/4 = 6` works there.

g++ warns about this even without `-Wall`: 'sizeof' on array function parameter 'arr' will return size of 'int*' [-Wsizeof-array-argument].

(c) Pass the number of elements as a second parameter, as in the instructor's `printData(int *d, const int n)`:

```
// print data
// parameter: arr - int *: pointer to data array
// parameter: n - const int: number of elements in array
void printData(int *arr, const int n) {
    cout << "n=" << n << ": ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
// call: printData(data, sizeof(data) / sizeof(data[0]));
```

This prints `n=6: 4 8 15 16 23 42`. A `std::vector` or the instructor's `Vector` class avoids the problem because it stores its own size.

Verified: `scratchpad/verify_bank_memory/ptr/decay.cpp` (exact code). `g++ -std=c++11 -Wall -Wextra` gave `decay.cpp:6:20: warning: 'sizeof' on array function parameter 'arr' will return size of 'int*' [-Wsizeof-array-argument]`, and plain `g++` `decay.cpp` gave the same warning. The program printed in `main`: 6 and `n=2: 4 8` (cat -A confirmed the trailing space). `clang++ -Wall` gave `-Wsizeof-array-argument` and `-Wsizeof-pointer-div` warnings. `decay_fix.cpp` compiled with no warnings and printed `n=6: 4 8 15 16 23 42`.

PTR-7 · output-prediction

Which swap functions actually swap the caller's variables? Give the exact output.

```
#include <iostream>
using namespace std;

void swapV(int a, int b) { int t = a; a = b; b = t; }
void swapP(int *a, int *b) { int t = *a; *a = *b; *b = t; }
void swapR(int &a, int &b) { int t = a; a = b; b = t; }
void swapX(int *a, int *b) { int *t = a; a = b; b = t; }

int main() {
    int x = 1, y = 2;
    swapV(x, y);    cout << "A: " << x << " " << y << endl;
    swapP(&x, &y);  cout << "B: " << x << " " << y << endl;
    swapR(x, y);    cout << "C: " << x << " " << y << endl;
    swapX(&x, &y);  cout << "D: " << x << " " << y << endl;
    int arr[3] = {7, 8, 9};
    swapR(arr[0], arr[2]);
    swapP(&arr[0], arr + 1);
    cout << "E: " << arr[0] << " " << arr[1] << " " << arr[2] << endl;
}
```

```
    return 0;
}
```

Also: does `swapR(3, 4);` compile?

Answer

```
A: 1 2
B: 2 1
C: 1 2
D: 1 2
E: 8 9 7
```

- **swapV (pass by value): does NOT work.** a and b are **copies** in swapV's stack frame. The copies are swapped and then disappear when the function returns, so x and y stay 1 2.
- **swapP (pass by pointer): works.** The function gets the **addresses** of x and y and changes them through `*a` and `*b`, giving 2 1. The call must pass addresses: `swapP(&x,&y)`.
- **swapR (pass by reference): works.** a and b are **aliases** of x and y. This is the instructor's `swap(int &a, int &b)` from sort.cpp. It swaps 2 1 back to 1 2, and the call looks like pass by value: `swapR(x,y)`.
- **swapX: does NOT work.** It swaps the local **copies of the pointers**, not the ints they point to. Nothing is dereferenced, so x and y are unchanged.
- **E: {7,8,9} → swapR(arr[0],arr[2]) gives {9,8,7} → swapP(&arr[0], arr+1) gives {8,9,7}.** (`arr + 1` is `&arr[1]`.)

`swapR(3, 4);` does **not compile**: error: cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'.

- A non-const reference must bind to an lvalue, a real variable or element that can be changed.
- A literal such as 3 is not one.
- A `const int &` parameter could accept 3, but then the function could not swap anything.

Verified: scratchpad/verify_bank_memory/ptr/swaps.cpp (exact code) compiled with `g++ -std=c++11 -Wall -Wextra` with no warnings. It printed A: 1 2, B: 2 1, C: 1 2, D: 1 2, E: 8 9 7. With `-DBAD (swapR(3, 4);)` it gave swaps.cpp:20:11: error: cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'.

PTR-8 · trace

Trace the program and give the exact output.

```
#include <iostream>
using namespace std;

int &bigger(int &a, int &b) {
    return (a > b) ? a : b;
}

int main() {
    int x = 5, y = 10;
    int &r = x;
    int *p = &y;
```

```

r++;
cout << "1: " << x << " " << r << endl;
r = y;
y = 20;
cout << "2: " << x << " " << r << " " << y << endl;
*p = *p + r;
cout << "3: " << y << endl;
p = &x;
*p = 7;
cout << "4: " << x << " " << r << " " << y << endl;
int **pp = &p;
**pp = 99;
cout << "5: " << x << " " << (&r == &x) << " " << (p == &x) << endl;
int &r2 = *p;
r2 -= 9;
cout << "6: " << r << endl;
bigger(x, y) = 0;
cout << "7: " << x << " " << y << endl;
return 0;
}

```

Also: why do `int &r;` and `int &r = nullptr;` not compile, while `int *p;` does?

Answer

```

1: 6 6
2: 10 10 20
3: 30
4: 7 7 30
5: 99 1 1
6: 90
7: 0 30

```

Step by step:

1. `r` is an **alias** of `x`, so `r++` makes `x = 6`. Printing `x` and `r` reads the same variable twice.
2. `r = y;` does **not** re-seat the reference. It **copies the value** 10 into `x`. Changing `y` to 20 later does not affect `x` (`x = 10`, `r = 10`, `y = 20`).
3. `p` points to `y`: `*p = 20 + 10`, so `y = 30`.
4. A pointer **can** be re-pointed: `p = &x;` `*p = 7;` gives `x = 7`, so `r = 7`. `y` is still 30.
5. `pp` is a pointer to a pointer. `**pp` is `*p`, which is `x`, so `x = 99`.
 - `&r == &x` is true: taking the address of a reference gives the address of the variable it refers to.
 - `p == &x` is true.
6. `r2` is another alias of `*p`, which is `x`: `99 - 9 = 90`, and `r` reads `x`.
7. `bigger` **returns a reference** to the larger variable (`x = 90 > y = 30`), so the assignment sets `x` to 0.

Compile rules: a reference must be **initialized** when it is declared and **cannot be null**.

- `int &r;` gives error: 'r' declared as reference but not initialized.
- `int &r = nullptr;` gives error: invalid initialization of non-const reference of type 'int&' from an rvalue of type 'std::nullptr_t'.

A pointer may be declared uninitialized (dangerous, see BUG-10) or set to `nullptr`, and it can be re-pointed later.

'Cannot be null' does not mean 'always valid': a reference can still **dangle** if the variable it refers to is destroyed, e.g. a reference to a local variable returned from a function.

Verified: scratchpad/verify_bank_memory/ptr/refs.cpp (exact code) compiled with `g++ -std=c++11 -Wall -Wextra` with no warnings. It printed 1: 6 6, 2: 10 10 20, 3: 30, 4: 7 7 30, 5: 99 1 1, 6: 90, 7: 0 30. `referr.cpp`: -DE1 gave `referr.cpp:3:10: error: 'r' declared as reference but not initialized`, and -DE2 gave `referr.cpp:6:14: error: invalid initialization of non-const reference of type 'int&' from an rvalue of type 'std::nullptr_t'.` -DP (`int *p;`) compiled.

MEM-9 · short-answer

(a) For each numbered item, give the memory region using the instructor's memory map: **stack, heap, globals (R/W), code (R/O), global constants (R/O)**. The program prints the addresses of the items. Order the **regions** from **highest to lowest** address and explain the order.

```
#include <iostream>
using namespace std;

int total = 5; // (1)
const int MAX = 100; // (2)

int *makeArray(int n) { // (3) the code of makeArray
    static int calls = 0; // (4)
    calls++;
    int *temp = new int[n]; // (5) temp (6) the n ints
    cout << "&calls " << &calls << "\n&temp " << &temp << "\n&n " << &n << endl;
    return temp;
}

int main(int argc, char *argv[]) { // (7) argc
    int l1 = 12; // (8)
    int arr1[10]; // (9)
    arr1[0] = l1;
    const char *msg = "Hello"; // (10) msg (11) "Hello"
    int *aptr = makeArray(10); // (12) aptr
    cout << "&total " << &total << "\n&MAX " << &MAX
        << "\nmakeArray " << (void *)makeArray
        << "\n&argc " << &argc << "\n&l1 " << &l1 << "\narr1 " << arr1
        << "\n&msg " << &msg << "\nmsg " << (const void *)msg
        << "\n&aptr " << &aptr << "\naptr " << aptr << endl;
    (void)argv;
    delete [] aptr;
    aptr = nullptr;
    return 0;
}
```

(b) Static vs automatic: what does this print?

```
#include <iostream>
using namespace std;

int g = 0;
int f() {
    static int s = 0;
    int a = 0;
    s++; a++; g++;
}
```

```

    return s * 100 + a * 10 + g;
}
int main() { cout << f() << endl; cout << f() << endl; cout << f() << endl; }

```

(c) Given `char str[] = "hello";` and `const char *ps = "hello";`, what do `sizeof(str)` `strlen(str)` `sizeof(ps)` `strlen(ps)` print? Are `str[0] = 'J';` and `ps[0] = 'J';` allowed?

Answer

(a) Regions

#	item	region
1	<code>total</code> (global)	globals (R/W) ; exists for the whole program (static allocation)
2	<code>MAX</code> (global const)	global constants (R/O)
3	body of <code>makeArray</code> (and of <code>main</code>)	code (R/O)
4	<code>static int calls</code>	globals (R/W) . It is a static local: visible only inside <code>makeArray</code> but stored in the global space, initialized once, and keeps its value between calls.
5	<code>temp</code> (the pointer variable)	stack , in <code>makeArray</code> 's frame; it disappears when <code>makeArray</code> returns
6	the 10 ints from <code>new int[n]</code>	heap , reached only through a pointer; lives until <code>delete []</code>
7	<code>argc</code> (parameter)	stack , <code>main</code> 's frame
8	<code>l1</code>	stack
9	<code>arr1</code> (local array)	stack , <code>main</code> 's frame
10	<code>msg</code> (the pointer)	stack
11	the literal <code>"Hello"</code>	read-only constants (R/O) , together with the global constants
12	<code>aptr</code>	stack . The pointer is on the stack; what it points to (item 6) is on the heap.

Address order observed (highest → lowest):

1. `main`'s frame (`arr1`, `&aptr`, `&msg`, `&l1`, `&argc`), around `0x7ffe...`
2. `makeArray`'s frame (`&temp`, `&n`)
3. heap block (`aptr`)
4. globals (`&calls`, `&total`)
5. R/O constants (`msg`, the `"Hello"` literal, and `&MAX`)
6. code (`makeArray`), around `0x55...`

Why:

- The **stack is at the top and grows down**. `makeArray` is called from `main`, so its frame is **below** `main`'s frame.
- The **heap** sits below the stack and grows up.
- The **globals**, **constants** and **code** come from the executable file and are at the bottom.

- The compiler chooses the order of the locals *inside* one frame, and of the items inside one region. clang++ put `&argc` highest in main's frame and `&MAX` above the literal, but the order of the regions was the same.
- The lecture picture draws `g.const` below the code. On Linux the read-only constants (`.rodata`) are placed just *above* the code (`.text`). Both are read-only, and both are below the globals.
- Exact addresses change on every run (address randomization); the relative order does not.

(b)

```
111
212
313
```

- `s` is **static** (global space, initialized once), so it counts 1, 2, 3.
- `a` is **automatic** (stack) and is re-created as 0 on every call, so it is always 1.
- `g` is a global and also counts 1, 2, 3.

(c) It prints `6 5 8 5`.

- `str` is a **local array** on the stack holding a copy of the 5 letters plus `'\0'` (6 bytes).
- `ps` is an 8-byte **pointer** to the literal, which is stored in read-only memory.

Writing to them:

- `str[0] = 'J';` is fine, and `str` becomes `Jello`.
- `ps[0] = 'J';` does **not compile**: error: assignment of read-only location `'* ps'`.
- If the `const` is cast away (`char *bad = (char*)ps; bad[0]='J';`), the program compiles but crashes with a **Segmentation fault**, because the literal is in R/O memory. Writing to `MAX` through a cast also gives a segmentation fault.

`std::string` is a C++ class that sizes itself to fit: after `string s2 = s1; s2[0]='J';`, `s1` is still `hello`.

Verified: `scratchpad/verify_bank_memory/regions/regions2.cpp` (exact code) compiled with no diagnostics under both `g++ -std=c++11 -Wall -Wextra -pedantic` and `clang++` with the same flags, and `valgrind -q --leak-check=full` was clean. The addresses were sorted numerically with a python helper.

- g++, high→low: `arr1 > &aptr > &msg > &l1 > &argc > &temp > &n > aptr > &calls > &total > msg > &MAX > makeArray`.
- clang++: `&argc > &l1 > arr1 > &msg > &aptr > &n > &temp > aptr > &calls > &total > &MAX > msg > makeArray`. The region order is the same.
- An earlier g++ run of the original version printed `arr1 0x7ffe2ebae470 ... &n 0x7ffe2ebae41c, aptr 0x55bd282742b0, &calls 0x55bcf9ba8154, &count 0x55bcf9ba8010, msg 0x55bcf9ba6026, &MAX 0x55bcf9ba6008, makeArray 0x55bcf9ba5209`.
- `readelf -S: .text 0x1120 < .rodata 0x2000 < .data 0x4000 < .bss 0x4040`.

Why the original code was changed: with `-pedantic` it gave ISO C++ forbids taking address of function `:::main'`. `cnt.cpp` (global count + using namespace std + `<algorithm>`) gave error:

reference to 'count' is ambiguous.

(b) statloc.cpp printed 111 / 212 / 313.

(c) cstr.cpp printed 6 5 8 5 and Jello hello hello Jello. -DE1 gave cstr.cpp:11:11: error: assignment of read-only location '* ps'. -DR1 (write to the literal through a cast) gave Segmentation fault, exit=139. romax.cpp (write to MAX through a cast) gave Segmentation fault, exit=139.

BUG-10 · find-the-bug

Four versions of a function that should return an array of the first n squares to the caller. For each one, say whether it is correct. If it is not, name the bug and say what happens.

```
int *f1(int n) { // version 1
    int arr[10];
    for (int i = 0; i < n; i++) arr[i] = i * i;
    return arr;
}
int *f2(int n) { // version 2
    int *arr = new int[n];
    for (int i = 0; i < n; i++) arr[i] = i * i;
    return arr;
}
int *f3(int n) { // version 3
    static int arr[10];
    for (int i = 0; i < n; i++) arr[i] = i * i;
    return arr;
}
int *f4(int n) { // version 4
    int *arr;
    for (int i = 0; i < n; i++) arr[i] = i * i;
    return arr;
}
// caller: int *p = fX(4); cout << p[0] << " " << p[1] << " " << p[2] << " " << p[3] << endl;
```

Answer

Version 1: BUG. It returns the address of a local variable (dangling pointer). `arr` is an automatic local array in `f1`'s **stack frame**. When `f1` returns, the frame is released, and the next function call (`cout << ...`) reuses that memory. This is undefined behavior.

- g++ warns address of local variable 'arr' returned [-Wreturn-local-addr].
- g++ actually compiled it to return a **null pointer** (a test printed `f1` returned 0), so `p[0]` crashed with a **Segmentation fault**.
- clang++ returned the old stack address and printed garbage (0 21851 -833974272 32550).

Either way the result is wrong.

Version 2: CORRECT. The block is on the **heap**, so it outlives the function. This is the instructor's `loadData` pattern. It prints 0 1 4 9. The **caller must** delete `[] p; p = nullptr;` or the block leaks.

Version 3: works, but with limits. A static local array is stored in the **global space**, so it still exists after the return, and it prints 0 1 4 9. The limits:

- There is only **one** array, shared by every call. After `int *a = f3(4); a[1] = 50; int *b = f3(4);`, `a == b` is true and `a[1]` is back to 1: the second call overwrote the caller's data. Writing `b[0] = 7` also changes `a[0]`.
- The size is fixed at 10, so `n > 10` writes past the end of the array.
- It must **not** be deleted: `delete [] a;` aborted with `free(): invalid pointer`.

Version 4: BUG. Uninitialized pointer. `arr` holds a garbage address, so `arr[i] = ...` writes to a random place in memory. g++ warns '`arr`' may be used uninitialized `[-Wmaybe-uninitialized]`. At run time it caused a **Segmentation fault** (exit 139).

Fix: allocate with `new` (version 2), and give every pointer a value (`nullptr` or the result of `new`) before using it. The caller frees the block with `delete []`.

Verified: `scratchpad/verify_bank_memory/bugs/retarr.cpp` (the four exact functions plus a driver choosing one by argv). g++ `-std=c++11 -Wall -Wextra` gave `retarr.cpp:6:12: warning: address of local variable 'arr' returned [-Wreturn-local-addr]` and `retarr.cpp:20:38: warning: 'arr' may be used uninitialized [-Wmaybe-uninitialized]`.

- Runs: f1 gave Segmentation fault, exit=139. f2 printed 0 1 4 9, exit=0. f3 printed 0 1 4 9, exit=0. f4 gave Segmentation fault, exit=139.
- clang++ build: f1 printed 0 21851 -833974272 32550 (exit 0), and f4 gave a Segmentation fault.
- `retarr2.cpp` printed f1 returned 0.
- `retarr3.cpp` (exact f3): `int *a = f3(4); a[1] = 50; int *b = f3(4);` printed 1 1 (`a==b`, `a[1]=1`). After `b[0] = 7`, `a[0]` printed 7. The -DDEL build (`delete [] a;`) printed `free(): invalid pointer`, Aborted, exit=134.

BUG-11 · find-the-bug

Find the **four** memory bugs, name each one, say what can happen, and fix it.

```
#include <iostream>
using namespace std;

int main() {                                // line 4
    int *a = new int[5];                    // line 5
    for (int i = 0; i < 5; i++) a[i] = i;
    a = new int[10];                        // line 7

    int *b = new int[3];                    // line 9
    delete b;                               // line 10

    int *c = new int(7);                     // line 12
    delete c;                               // line 13
    cout << *c << endl;                     // line 14

    int *d = new int[4];                     // line 16
    int *e = d;
    delete [] d;                            // line 18
    delete [] e;                            // line 19

    delete [] a;
```

```
    return 0;
}
```

Answer

- Line 7: memory leak.** `a` is pointed at a new block before the old 5-int block is freed. The only pointer to the first block is lost, so those 20 bytes can never be deleted.
 - `valgrind`: 20 bytes in 1 blocks are definitely lost (allocated at line 5).
 - **Fix:** `delete [] a;` before `a = new int[10];`.
- Line 10: delete used on memory from new[]**. Memory from `new[]` must be freed with `delete []`; mixing them is undefined behavior. For an array of objects it typically runs only one destructor and can crash (see OBJ-14).
 - `g++` warns 'void operator delete(void*)' called on pointer returned from a mismatched allocation function [-Wmismatched-new-delete].
 - `valgrind`: Mismatched free() / delete / delete [].
 - **Fix:** `delete [] b;`.
- Line 14: use after deallocation (dangling pointer).** `c` still holds the address, but the block was returned to the heap at line 13. Reading it is undefined; it printed a different garbage number on each run (e.g. 1668567984, 1620524851).
 - `valgrind`: Invalid read of size 4 at line 14.
 - **Fix:** use `*c` before the delete, then set `c = nullptr;`.
- Line 19: double delete.** `e` and `d` point to the **same** block (copying a pointer does not copy the block), so the block is freed twice. The program aborts with `free(): double free detected in tcache 2 (exit 134)`.
 - `valgrind`: Invalid free() / delete / delete[] / realloc() at line 19.
 - **Fix:** delete the block once (remove line 19) and set both pointers to `nullptr`.

With the four fixes the program prints 7, and `valgrind` reports All heap blocks were freed -- no leaks are possible and ERROR SUMMARY: 0 errors.

Rules: every `new` has exactly one `delete`, and every `new[]` has exactly one `delete []`. Set the pointer to `nullptr` after deleting.

Verified: `scratchpad/verify_bank_memory/bugs/fourbugs.cpp` (exact code, line numbers match). `g++ -std=c++11 -g -Wall -Wextra` gave only `fourbugs.cpp:10:12: warning: 'void operator delete(void*)' called on pointer returned from a mismatched allocation function [-Wmismatched-new-delete]`.

Three runs printed 1668567984, 1620524851 and 1617182220, each followed by `free(): double free detected in tcache 2 / Aborted with exit=134`.

`valgrind --leak-check=full` reported:

- Mismatched free() / delete / delete [] (fourbugs.cpp:10, allocated at :9)
- Invalid read of size 4 (fourbugs.cpp:14, freed at :13, allocated at :12)
- Invalid free() / delete / delete[] / realloc() (fourbugs.cpp:19, freed at :18, allocated at :16)
- 20 bytes in 1 blocks are definitely lost (fourbugs.cpp:5)
- ERROR SUMMARY: 4 errors from 4 contexts

fourbugs_fixed.cpp printed 7, and valgrind gave All heap blocks were freed -- no leaks are possible and ERROR SUMMARY: 0 errors from 0 contexts.

BUG-12 · find-the-bug

The program below is compiled on Linux (Ubuntu) with a plain `g++ fillSum.cpp -o fillSum`. On Ubuntu, g++ turns on the stack protector by default. What does it print, and why? What is the bug? What happens if it is compiled with `-fno-stack-protector`?

```
#include <iostream>
using namespace std;

int fillSum() {
    int arr[10];
    int sum = 0;
    for (int i = 0; i <= 10; i++) {
        arr[i] = i;
        sum += arr[i];
    }
    cout << "sum = " << sum << endl;
    return sum;
}

int main() {
    fillSum();
    cout << "back in main" << endl;
    return 0;
}
```

Answer

Output:

```
sum = 55
*** stack smashing detected ***: terminated
Aborted
```

back in main is **never printed** (exit status 134).

Bug: the loop uses `i <= 10` instead of `i < 10`. `arr` has indices 0–9, so `arr[10] = 10` writes one int **past the end of a local array**, into another part of `fillSum`'s **stack frame**.

Why it prints sum = 55 first:

- C++ does not check array bounds, so the bad write does not fail when it happens.
- g++ protects functions that have local arrays with a **stack canary**: a guard value stored in the frame just above the local array, below the saved frame pointer and return address.
- In this build `arr` starts at `rbp-48` and the canary is at `rbp-8`, which is exactly `arr[10]`, so the write overwrote the canary.
- The canary is checked only when the function **returns**, so the `cout` inside `fillSum` still runs ($0+1+\dots+10 = 55$).
- At the return the check fails and the program aborts.

This is the **"stack smashing"** marked on the 8/20 memory map: a local array overflowing into the rest of the frame (other locals, frame pointer, return address).

With -fno-stack-protector: the program printed `sum = 55` and `back in main` and exited normally. The bug is still there; it just was not detected.

- Without the canary, `rbp-8` holds the loop counter `i`. So `arr[10] = i` wrote 10 into `i` itself when `i` was already 10, and nothing visible changed.
- If the body is changed to `arr[i] = 0;`, the same write resets `i` to 0 and the loop never ends.
- In another program such a write could corrupt a different variable or the return address.

Warnings:

- The compiler gave **no warning** with `-Wall -Wextra` at the default optimization level.
- With `-O2`, g++ warns `iteration 10 invokes undefined behavior [-Waggressive-loop-optimizations]`.
- clang++ on this Linux system does not turn the protector on by default; its build printed `back in main`.

Fix: `for (int i = 0; i < 10; i++)`, or better, loop to the array's size.

Verified: `scratchpad/verify_bank_memory/bugs/fillSum.cpp` (exact code).

- A plain `g++ fillSum.cpp -o fs` build and a `g++ -std=c++11 -Wall -Wextra` build (no warnings) both printed `sum = 55, *** stack smashing detected ***: terminated, Aborted, exit=134`.
- `g++ -Q --help=common` shows `-fstack-protector-strong [enabled]`.
- `g++ -S`, protector on: `movq %rax, -8(%rbp)` (canary) and array stores `movl %edx, -48(%rbp,%rax,4)`, so `arr[10] = rbp-8`.
- `-fno-stack-protector` assembly: `cmpl $10, -8(%rbp)` (`i` at `rbp-8`) and array at `-48(%rbp,%rax,4)`.
- The `-fno-stack-protector` build printed `sum = 55` and `back in main, exit=0`.
- `fillZero.cpp` (`arr[i] = 0;`, `-fno-stack-protector`) had to be killed by `timeout 3`, `exit=124` (infinite loop).
- `-O2` gave `fillSum.cpp:8:16: warning: iteration 10 invokes undefined behavior [-Waggressive-loop-optimizations]` and printed `back in main`.
- The clang++ build printed `sum = 55, back in main, exit=0`.

BUG-13 · find-the-bug

This is the end of `main()` in the instructor's `sort.cpp`, where `loadData` does `int *temp = new int[n];` `for(...)` `std::cin >> temp[i];` `return temp;`:

```
int *data = nullptr;
data = loadData(num);
if (data==nullptr) {
    std::cerr << "Error reading data\n";
    return -1;
}
...
bubbleSort(data,num);
```

```

...
printData(data,num);
std::cout << "++++++++++++++++++++++++++++++++++++\n";
std::cout << "number: " << num << " took " << end-start << std::endl;

return 0;
}

```

(a) What memory bug does the program have? How many bytes does valgrind report for `./sort 5 < data.txt`? How do you fix it? (b) Can the `if (data==nullptr)` branch ever run? What really happens for `./sort -5`, or when there is not enough memory? How would you make the check meaningful?

Answer

(a) Memory leak. `loadData` allocates `n` ints on the **heap**, and its comment says "user must deallocate". `main` never calls `delete [] data`. valgrind shows:

```

in use at exit: 20 bytes in 1 blocks
total heap usage: 4 allocs, 3 frees, ...
20 bytes in 1 blocks are definitely lost ...
  by loadData(int) (sort.cpp:14)
  by main (sort.cpp:67)

```

- That is 5 ints × 4 bytes = 20. The lost amount is 4·n bytes: `./sort 3` loses 12 and `./sort 8` loses 32.
- The OS takes the memory back when the program exits, but it is still a leak. In a long-running program, or if `loadData` were called in a loop, the lost memory would keep growing.

Fix, before `return 0;`:

```

delete [] data;    // allocated with new[] in loadData -> delete []
data = nullptr;    // no dangling pointer

```

valgrind then reports 4 allocs, 4 frees and All heap blocks were freed -- no leaks are possible.

(b) No. A plain `new` never returns `nullptr`. On failure it **throws an exception**, which ends the program before the `if` is reached:

- `./sort -5` gives `terminate` called after throwing an instance of '`std::bad_array_new_length`' (Aborted, exit 134).
- Asking for 2,000,000,000 ints with memory limited (`ulimit -v 500000`) gives `terminate` called after throwing an instance of '`std::bad_alloc`' (Aborted).

To make the check meaningful:

1. **Check the count first** in main: `if (num <= 0) { std::cerr << "Number of values must be positive\n"; return -1; }`. Do not rely on `nothrow` for this case: with g++ 13, `new (std::nothrow) int[-5]` **still throws** `std::bad_array_new_length` (clang++ returned `nullptr` instead).
2. **Allocate with `nothrow` and stop at once in `loadData`:**

```

#include <new>
...
int *temp = new (std::nothrow) int[n];    // nullptr if out of memory
if (temp == nullptr) return nullptr;      // never fill a null pointer

```

The early return matters. With only the `new (std::nothrow)` change, the loop `std::cin >> temp[i]` wrote through the null pointer, and the program crashed with a **Segmentation fault** before main's check ran.

With both changes:

- the low-memory test printed `Error reading data (exit status 255);`
- `./sort -5` printed `Number of values must be positive.`

Alternatively, keep the plain `new` and catch `std::bad_alloc` with try/catch.

Verified: In `scratchpad/verify_bank_memory/`, built `g++ -std=c++11 -g -Wall -Wextra sort.cpp support.cpp -o sort_g`.

- `valgrind --leak-check=full ./sort_g 5 < data.txt` gave in use at exit: 20 bytes in 1 blocks, total heap usage: 4 allocs, 3 frees, 81,940 bytes allocated, 20 bytes in 1 blocks are definitely lost ... by `loadData(int)` (`sort.cpp:14`) by `main` (`sort.cpp:67`).
- `n=3` gave definitely lost: 12 bytes; `n=8` gave definitely lost: 32 bytes.
- `sort_fixed.cpp` gave 4 allocs, 4 frees, All heap blocks were freed -- no leaks are possible, ERROR SUMMARY: 0 errors.
- `./sort -5 < data.txt` gave terminate called after throwing an instance of `'std::bad_array_new_length'`, exit=134.
- `(ulimit -v 500000; ./sort 2000000000 < data.txt)` gave terminate called after throwing an instance of `'std::bad_alloc'`, exit=134.
- `sort_nothrow_nocheck.cpp` (only `new (std::nothrow)`) in the same test gave Segmentation fault, exit=139.
- `sort_nothrow.cpp` (`nothrow + if (temp == nullptr) return nullptr;`) printed `Error reading data`, exit=255. With `-5` its `g++` build still threw `std::bad_array_new_length` (exit 134), while its `clang++` build printed `Error reading data` (exit 255).
- `sort_checked.cpp` (plus the `num <= 0` check) printed `Number of values must be positive` (exit 255) for `-5`, `Error reading data` (exit 255) in the low-memory test, and ran normally for 5.

OBJ-14 · output-prediction

Give the exact output. How many times are the constructor and the destructor called? Is anything leaked?

```
#include <iostream>
using namespace std;

int nextId = 1;

class Box {
private:
    int id;
public:
    Box() : id(nextId++) { cout << "+" << id << " "; }
    virtual ~Box() { cout << "-" << id << " "; }
};

int main() {
    cout << "A: "; Box b1;                cout << endl;
```

```

cout << "B: "; Box *p = new Box;      cout << endl;
cout << "C: "; Box *arr = new Box[3];  cout << endl;
cout << "D: "; Box local[2];           cout << endl;
cout << "E: "; Box *ptrs[3];           cout << endl;
cout << "F: "; delete p;               cout << endl;
cout << "G: "; delete [] arr;          cout << endl;
cout << "H: "; { Box tmp; }            cout << endl;
cout << "I: "; Box *q = new Box;       cout << endl;
(void)ptrs; (void)q; (void)local;
cout << "end of main" << endl;
return 0;
}

```

What would change if line G were written `delete arr;`?

Answer

```

A: +1
B: +2
C: +3 +4 +5
D: +6 +7
E:
F: -2
G: -5 -4 -3
H: +8 -8
I: +9
end of main
-7 -6 -1

```

- A: `b1` is an object on the **stack**; its constructor runs at the declaration.
- B: `new Box` makes **one object on the heap**, calling the constructor once. `p` itself is on the stack.
- C: `new Box[3]` calls the default constructor **once per element**, in order: 3, 4, 5.
- D: a local array of 2 objects on the stack: 6, 7.
- E: `Box *ptrs[3]` is an array of 3 **pointers**. No `Box` objects are created, so no constructor runs.
- F: `delete p` runs the destructor of object 2, then frees the memory.
- G: `delete [] arr` runs the destructor for **each** element in **reverse** order (5, 4, 3).
- H: `tmp` is local to the inner block `{ }`, so it is created and destroyed there.
- I: object 9 is created on the heap and **never deleted**.
- At `return 0;` the stack objects still alive are destroyed in reverse order of construction: `local[1]` (7), `local[0]` (6), `b1` (1). Heap objects are never destroyed automatically; only `delete` destroys them.

Counts: constructor **9** times, destructor **8** times.

`Box 9` is a **memory leak**: `valgrind` reports 16 bytes in 1 blocks are definitely lost at the `new Box` of line I. `sizeof(Box)` is 16 because `virtual` adds a hidden vtable pointer (8 bytes) to the `int` (4 bytes) plus 4 bytes of padding. Without `virtual` it is 4.

If G were `delete arr;`: this is undefined behavior, because memory from `new[]` needs `delete []`. In the g++ 13 test:

- The destructor ran for **`arr[0]` only** (`Box 3`). Then the program crashed with `munmap_chunk(): invalid pointer` (Aborted, exit 134), and Boxes 4 and 5 were never destroyed.
- On a normal run the screen shows lines A–F and then the error message. The text `G: -3` was still in `cout`'s buffer (no `endl` yet), so it is lost when the program aborts. It becomes visible only with

unbuffered output (cout << unitbuf;).

- g++ gave **no warning** for this class, because its destructor is virtual.
- clang++ warns 'delete' applied to a pointer that was allocated with 'new[]'; did you mean 'delete[]'?.
- g++ warns -Wmismatched-new-delete only when the destructor is not virtual (and for the int array in BUG-11).

Verified: scratchpad/verify_bank_memory/objs/box.cpp (exact code). g++ -std=c++11 -g -Wall -Wextra -pedantic gave no warnings. The output matched the listing exactly; cat -A showed the trailing spaces and that the last line -7 -6 -1 has no newline. The clang++ build gave the same output.

valgrind --leak-check=full ./box gave 16 bytes in 1 blocks are definitely lost ... main (box.cpp:23) and ERROR SUMMARY: 1 errors. sz.cpp printed 16 4 (virtual vs non-virtual destructor).

boxG.cpp (the question code with line G changed to delete arr;):

- g++ -Wall -Wextra gave no warning.
- The run printed lines A: through F: -2, then munmap_chunk(): invalid pointer, Aborted, exit=134. The same happened under a pseudo-terminal (script -qc ./boxG).
- boxG2.cpp (with cout << unitbuf;) printed G: -3 munmap_chunk(): invalid pointer, exit=134.
- clang++ warned boxG2.cpp:21:20: warning: 'delete' applied to a pointer that was allocated with 'new[]'; did you mean 'delete[]'? [-Wmismatched-new-delete].
- boxNV.cpp (non-virtual destructor): g++ warned 'void operator delete(void*)' called on pointer returned from a mismatched allocation function [-Wmismatched-new-delete].

OBJ-15 · find-the-bug

(9/22 lecture: shallow vs deep copy.)

```
#include <iostream>
using namespace std;

class Student {
private:
    int s_id;
    int *data;           // heap array of 3 grades
public:
    Student(int id) : s_id(id), data(new int[3]) {
        for (int i = 0; i < 3; i++) data[i] = 0;
    }
    virtual ~Student() { delete [] data; }
    void setGrade(int i, int g) { data[i] = g; }
    int getGrade(int i) { return data[i]; }
};

int main() {
    Student s1(10);
    s1.setGrade(0, 90);
    Student s2 = s1;
    s2.setGrade(0, 55);
    cout << s1.getGrade(0) << " " << s2.getGrade(0) << endl;
}
```

```
    return 0;
}
```

(a) What does this program print, and what goes wrong at the end? Fix it. (b) If the class also had an array member `int arr[100];`, would `arr` be shared by `s1` and `s2` after the copy? (c) After adding a copy constructor, is `Student s1(10), s2(20); s2 = s1;` safe? If not, what else must be written?

Answer

(a) Output:

```
55 55
free(): double free detected in tcache 2
Aborted
```

Why: the class has no copy constructor, so `Student s2 = s1;` uses the compiler's default copy, which copies member by member. That is a **shallow copy**:

- `s2.data` gets the **same address** as `s1.data`, so both objects share **one heap block**.
- Setting `s2`'s grade therefore also changes `s1`'s grade (55 55).
- At the end of `main` both destructors run `delete [] data` on the **same** block. That is a **double delete**, and the program aborts (valgrind: `Invalid free() in Student::~~Student()`).

Fix: a copy constructor that makes a deep copy (instructor: `Student(Student &) → "deep copy"`). The copy gets its own heap block:

```
// copy constructor - deep copy
Student(const Student &other) : s_id(other.s_id), data(new int[3]) {
    for (int i = 0; i < 3; i++) data[i] = other.data[i];
}
```

The program then prints `90 55` and exits normally, and valgrind reports `All heap blocks were freed -- no leaks are possible`.

(b) No. An array member is **inside the object**, so the default copy copies all 100 ints.

- In the test, after the copy `s1.arr[0]` stayed 1 while `s2.arr[0]` was set to 2, and `&s1.arr[0] == &s2.arr[0]` was 0.
- Only **pointer** members are shared: `s1.data == s2.data` was 1, and writing `s2.data[0] = 2` changed `s1.data[0]` too.

(c) No. `s2 = s1;` on an object that already exists calls the **assignment operator** (`operator=`), not the copy constructor, and the default assignment operator is also shallow.

- The test printed `55 55`, then `free(): double free detected`.
- valgrind also found `definitely lost: 12 bytes`: `s2`'s original block, lost when its pointer was overwritten.
- `g++ -Wextra` warns implicitly-declared `'Student& Student::operator=(const Student&)'` is deprecated `[-Wdeprecated-copy]`.

Add a deep-copy assignment operator:

```
// assignment operator - deep copy
Student &operator=(const Student &other) {
    if (this != &other) {                // s1 = s1 must not break
        s_id = other.s_id;
        for (int i = 0; i < 3; i++) data[i] = other.data[i];
    }
    return *this;
}
```

Both objects already own a 3-int block, so the values are copied into s2's own block. With it the program printed 90 55, and valgrind was clean. A class that owns heap memory needs a destructor, a copy constructor **and** an assignment operator (the "rule of three").

Note on the 9/22 lecture: the board writes `s2 = s1` next to "Copy Constructor". On the exam, be precise:

- The **copy constructor** runs when a **new** object is created from another: `Student s2 = s1;`, `Student s2(s1);`, or passing/returning a `Student` by value.
- `s2 = s1;` on an existing `s2` calls `operator=`.

Verified: scratchpad/verify_bank_memory/objs/shallow.cpp (exact code).

- `g++ -std=c++11 -g -Wall -Wextra -pedantic` gave no warnings. The program printed 55 55, then `free(): double free detected in tcache 2, Aborted, exit=134`.
- `valgrind` gave `Invalid free() / delete / delete[] / realloc() at Student::~~Student() (shallow.cpp:18), called from main (shallow.cpp:30), with the block allocated at shallow.cpp:24`.
- The `-DFIX` build (copy constructor) printed 90 55 with `exit=0`. `valgrind` gave All heap blocks were freed -- no leaks are possible and `ERROR SUMMARY: 0 errors`.

`members.cpp` printed 1 2 0 (`s1.arr[0]`, `s2.arr[0]`, same address?) and 2 2 1 (`s1.data[0]`, `s2.data[0]`, same pointer?); `valgrind` was clean.

`assign.cpp` (copy constructor only, `s2 = s1`):

- It gave `assign.cpp:34:10: warning: implicitly-declared 'Student& Student::operator=(const Student&)' is deprecated [-Wdeprecated-copy] with -Wextra`, and no warning with `-Wall` alone.
- It printed 55 55 and `free(): double free detected in tcache 2, exit=134`.
- `valgrind` gave `Invalid free()` and definitely lost: 12 bytes in 1 blocks.
- The `-DASSIGN` build (`operator=` added) printed 90 55 with `exit=0`. `valgrind` gave All heap blocks were freed -- no leaks are possible and `ERROR SUMMARY: 0 errors`.

Objects, classes, inheritance, polymorphism, copying and vectors

OBJ-01 · write-code

(a) **Write the class described by the UML box below.** Put the declaration in `sensor.h` (with an include guard) and the definition in `sensor.cpp`. The default constructor sets `id` to -1, `location` to "none" and no readings. `getAverage()` returns 0 if there are no readings. `print` writes exactly one line, in the form `Sensor <id> (<location>): <count> readings, average <average>`, and returns the stream.

```
+-----+
|               Sensor               |
+-----+
| - id : int                         |
| - location : string                |
| - total : double                   |
| - count : int                      |
+-----+
| + Sensor()                        |
| + Sensor(id : int, location : string) |
| + ~Sensor()                       |
| + getId() : int                   |
| + getLocation() : string          |
| + getAverage() : double           |
| + setLocation(location : string) : void |
| + addReading(value : double) : void  |
| + print(out : ostream&) : ostream& |
+-----+
```

(b) Give the command to compile `main.cpp` and your class into an executable called `sensor`. (c) What does this `main.cpp` print?

```
#include <iostream>
#include <string>
using namespace std;
#include "sensor.h"

int main() {
    Sensor s1;
    s1.print(cout);
    Sensor *s2 = new Sensor(7, "Lab 101");
    s2->addReading(20.0);
    s2->addReading(21.5);
    s2->addReading(23.0);
    s2->setLocation("Lab 102");
    s2->print(cout);
    cout << s2->getId() << " " << s2->getLocation() << " " << s2->getAverage() << endl;
    delete s2;
    return 0;
}
```

Answer

(a) **sensor.h: the declaration.** It has an include guard, private data (-) and public methods (+).

```
#ifndef SENSOR_H_
#define SENSOR_H_
```

```

#include <iostream>
#include <string>

class Sensor {
private:
    int id;                // sensor id
    std::string location;  // where the sensor is installed
    double total;          // sum of all readings
    int count;             // number of readings

public:
    // constructors
    Sensor();
    Sensor(int, std::string);

    // destructor
    virtual ~Sensor();

    // getters
    int getId();
    std::string getLocation();
    double getAverage();

    // setters
    void setLocation(std::string);
    void addReading(double);

    // print the sensor
    std::ostream& print(std::ostream&);
};

#endif

```

sensor.cpp: the definition. Every method is written with the `Sensor::` scope, and the constructors use initializer lists.

```

#include "sensor.h"

// default constructor - initializer list sets the defaults
Sensor::Sensor() : id(-1), location("none"), total(0.0), count(0) {}

// param: id : int - sensor id
// param: location : string - where the sensor is
Sensor::Sensor(int id, std::string location)
    : id(id), location(location), total(0.0), count(0) {}

// destructor - nothing on the heap, nothing to do
Sensor::~Sensor() {}

int Sensor::getId() { return id; }

std::string Sensor::getLocation() { return location; }

// return: double - average of the readings, 0 if there are none
double Sensor::getAverage() {
    if (count == 0)
        return 0.0;           // avoid division by zero
    return total / count;
}

// param: location : string - new location
void Sensor::setLocation(std::string location) {

```

```

    this->location = location;    // this-> : the member, not the parameter
}

// param: value : double - new reading
void Sensor::addReading(double value) {
    total += value;
    count++;
}

// param: out : ostream& - output stream
// return: ostream& - the same stream (allows chaining)
std::ostream& Sensor::print(std::ostream &out) {
    out << "Sensor " << id << " (" << location << "): "
        << count << " readings, average " << getAverage() << std::endl;
    return out;
}

```

What a grader looks for:

- an include guard
- private data and public methods
- initializer lists (in `id(id)` the first name is the member and the second is the parameter)
- `Sensor::` on every method definition
- `this->location = location` (the parameter has the same name as the member)
- a virtual destructor
- `print` returning `out`
- a guard against dividing by zero

(b) `g++ main.cpp sensor.cpp -o sensor`, then `./sensor`. Both `.cpp` files go on the compile line. The `.h` file is not listed because it is `#include d`.

(c) Output

```

Sensor -1 (none): 0 readings, average 0
Sensor 7 (Lab 102): 3 readings, average 21.5
7 Lab 102 21.5

```

- `s1` is on the stack and was made by the default constructor.
- `s2` is on the heap, so the code uses `->` and must `delete s2`.
- The average is $(20 + 21.5 + 23) / 3 = 64.5 / 3 = 21.5$.
- The location printed is "Lab 102" because `setLocation` ran before `print`.

Verified: `Dir verify_bank_oop/q01/`.

- `g++ -std=c++11 -Wall -Wextra -pedantic main.cpp sensor.cpp -o sensor:rc=0`, no warnings.
`clang++ 18` with the same flags: `rc=0`, no warnings.
- `./sensor` printed:

```

Sensor -1 (none): 0 readings, average 0
Sensor 7 (Lab 102): 3 readings, average 21.5
7 Lab 102 21.5

```

- `valgrind --leak-check=full ./sensor`: All heap blocks were freed -- no leaks are possible, ERROR SUMMARY: 0 errors from 0 contexts.
- With `-Wshadow`, g++ notes only that the constructor parameters shadow the members, which is harmless here.

OBJ-02 · find-the-bug

The files below do not compile. For each numbered line, say whether it compiles. If it does not, explain why and give a fix.

```
// book.h
#ifndef BOOK_H_
#define BOOK_H_
#include <iostream>
#include <string>
class Book {
    private:
        std::string title;
        double price;
        bool validPrice(double);
    public:
        Book(std::string, double);
        virtual ~Book();
        double getPrice();
        void setPrice(double);
        std::ostream& print(std::ostream&);
};
#endif
```

```
// book.cpp
#include "book.h"
Book::Book(std::string t, double p) : title(t), price(p) {}
Book::~Book() {}
double Book::getPrice() { return price; }
std::ostream& Book::print(std::ostream &out) {
    out << title << " $" << price << std::endl;
    return out;
}

bool Book::validPrice(double p) { // line 11
    return p >= 0.0;
}

void setPrice(double p) { // line 15
    if (validPrice(p)) // line 16
        price = p; // line 17
}
```

```
// main.cpp
#include <iostream>
#include "book.h"
using namespace std;
int main() {
    Book b("C++ Primer", 59.99); // line 6
    b.price = 10.0; // line 7
    cout << b.title << endl; // line 8
    cout << b.getPrice() << endl; // line 9
}
```

```

    b.setPrice(45.0);           // line 10
    if (b.validPrice(-1.0))    // line 11
        cout << "valid" << endl;
    Book c;                    // line 13
    b.print(cout);              // line 14
    return 0;
}

```

Answer

book.cpp

- **Line 11: OK.** A private helper method is defined like any other member, with `Book::`.
- **Lines 15-17: error.** `setPrice` is defined **without** `Book::`. It is therefore a new global function, not the member function. It has no object and no `this`, so `price` and `validPrice` are unknown names:
 - error: 'validPrice' was not declared in this scope (line 16)
 - error: 'price' was not declared in this scope (line 17)

Fix: `void Book::setPrice(double p) { if (validPrice(p)) price = p; }`

If the body had used no members, `book.cpp` would compile. The **linker** would then fail instead, because the member `Book::setPrice` was declared and called (main line 10) but never defined:

```
undefined reference to `Book::setPrice(double)'
```

main.cpp

line	result	reason / fix
6	OK	the public constructor exists
7 <code>b.price = 10.0;</code>	error 'double Book::price' is private within this context	private data can only be used by Book's own methods. Use <code>b.setPrice(10.0);</code>
8 <code>b.title</code>	error 'std::string Book::title' is private within this context	use <code>b.print(cout)</code> or add a <code>getTitle()</code> getter
9, 10	OK	public methods
11 <code>b.validPrice(-1.0)</code>	error 'bool Book::validPrice(double)' is private within this context	a private helper can only be called by Book's own methods (<code>setPrice</code> calls it)
13 <code>Book c;</code>	error no matching function for call to 'Book::Book()'	the class declares a constructor, so the compiler no longer supplies a default one. Use <code>Book c("Untitled", 0.0);</code> or add <code>Book();</code> to the class
14	OK	

Private members still exist inside `b`. Information hiding is checked by the **compiler**: it only controls which code may name the members.

Corrected main.cpp


```

int main() {
    Book b("C++ Primer", 59.99);
    b.setPrice(10.0);           // fixes line 7
    b.print(cout);              // fixes line 8
    cout << b.getPrice() << endl; // line 9
    b.setPrice(45.0);           // line 10
    b.setPrice(-1.0);           // fixes line 11: validPrice runs inside setPrice, -1 is
    rejected
    Book c("Untitled", 0.0);     // fixes line 13
    b.print(cout);              // line 14
    c.print(cout);
    return 0;
}

```

Output:

```

C++ Primer $10
10
C++ Primer $45
Untitled $0

```

The price stays 45 because `setPrice(-1.0)` is rejected by `validPrice`.

Verified: Dir `verify_bank_oop/q02/` (the files exactly as in the question).

- `g++ -std=c++11 -Wall -Wextra -c book_bad.cpp:`

```

book_bad.cpp:16:9: error: 'validPrice' was not declared in this scope
book_bad.cpp:17:9: error: 'price' was not declared in this scope

```

- `g++ -std=c++11 -Wall -Wextra -c main_bad.cpp:`

```

main_bad.cpp:7:7: error: 'double Book::price' is private within this context
main_bad.cpp:8:15: error: 'std::string Book::title' is private within this context
main_bad.cpp:11:21: error: 'bool Book::validPrice(double)' is private within this context
main_bad.cpp:13:10: error: no matching function for call to 'Book::Book()'

```

- Linker variant (a free `setPrice` whose body uses no members, main calls `b.setPrice(45.0)`):
`main_link.cpp:(.text+0x8c): undefined reference to Book::setPrice(double)', collect2: error: ld returned 1 exit status`.`
- Fixed `book.cpp` plus the corrected main: `g++ -std=c++11 -Wall -Wextra -pedantic main.cpp book.cpp -o book` gave 0 warnings. It printed `C++ Primer $10 / 10 / C++ Primer $45 / Untitled $0`. `valgrind`: All heap blocks were freed, ERROR SUMMARY: 0 errors.

OBJ-03 · output-prediction

(a) What does this program print? Explain each line. (b) What happens if you add `cout << q->getId() << endl;` before `delete q;`? (c) What changes if the word `virtual` is removed from `print()` in `Person`?

```

#include <iostream>
#include <string>
using namespace std;

```

```

class Person {
protected:
    string name;
public:
    Person(string n) : name(n) {}
    virtual ~Person() {}
    void who() { cout << "Person " << name << endl; } // NOT virtual
    virtual void print() { cout << "Person::print " << name << endl; } // virtual
};

class Student : public Person {
private:
    int s_id;
public:
    Student(string n, int id) : Person(n), s_id(id) {}
    void who() { cout << "Student " << name << endl; }
    void print() { cout << "Student::print " << name << " " << s_id << endl; }
    int getId() { return s_id; }
};

int main() {
    Student s("Ann", 42);
    Person *p = &s;
    Person &r = s;
    Person c = s;

    s.who(); // 1
    p->who(); // 2
    p->print(); // 3
    r.print(); // 4
    c.print(); // 5

    Person *q = new Student("Bob", 7);
    q->print(); // 6
    q->who(); // 7
    delete q;
    return 0;
}

```

Answer

(a) Output

```

Student Ann
Person Ann
Student::print Ann 42
Student::print Ann 42
Person::print Ann
Student::print Bob 7
Person Bob

```

1. `s.who()`: `s` is a `Student`, so `Student's who` runs.
2. `p->who()`: `who` is **not virtual**, so the compiler picks the function from the **pointer type** (`Person*`). This is static binding, decided at compile time.
3. `p->print()`: `print` is **virtual**, so the function is picked at run time from the **real object**, which is a `Student`. This is polymorphism, the lecture's `Person *p = s; p->print(); // Student version`.
4. `r.print()`: a reference to a base class behaves like a base pointer here, so it is also the `Student` version.

5. `c.print()`: `c` is a real `Person` object. `Person c = s`; copied only the `Person` part of `s` ("slicing"), so there is no `Student` left and `Person`'s `print` runs.
6. `q->print()`: virtual, so the version is chosen from the heap object, a `Student`.
7. `q->who()`: non-virtual, so the `Person` version runs.

(b) Compile error: `'class Person' has no member named 'getId'`. Through a `Person*` the compiler only lets you call what `Person` declares. Polymorphism only changes *which version* of a method declared in the base class runs. To call it, use a `Student` pointer: `static_cast<Student*>(q)->getId()` (we know `q` points to a `Student`) prints 7. Alternatively, declare `getId` in `Person`.

(c) Every `print` call then uses the type of the pointer, reference or variable, which is `Person` each time. Lines 3-6 become `Person::print Ann`, `Person::print Ann`, `Person::print Ann`, `Person::print Ann`. Lines 1, 2 and 7 do not change.

Verified: `Dir verify_bank_oop/q03/`.

- `g++ -std=c++11 -Wall -Wextra -pedantic -DVIRT=virtual poly.cpp -o poly && ./poly` gave no warnings and printed exactly the 7 lines above. `clang++` output was identical (diff empty).
`valgrind`: All heap blocks were freed, ERROR SUMMARY: 0 errors.
- With `-DBAD` (the `getId` line added): `poly.cpp:41:16: error: 'class Person' has no member named 'getId'`.
- `fixb.cpp` using `static_cast<Student*>(q)->getId()` compiled cleanly and printed 7.
- `-DVIRT=` (virtual removed from `print`) printed:

```
Student Ann
Person Ann
Person::print Ann
Person::print Ann
Person::print Ann
Person::print Bob
Person Bob
```

OBJ-04 · output-prediction

(a) What does this program print? (b) What changes in section B if `virtual` is removed from `~Person()`, and what goes wrong?

```
#include <iostream>
using namespace std;

class Address {
public:
    Address() { cout << "Address()" << endl; }
    ~Address() { cout << "~Address()" << endl; }
};

class Person {
public:
    Person() { cout << "Person()" << endl; }
    virtual ~Person() { cout << "~Person()" << endl; }
};
```

```

class Student : public Person {
private:
    Address home;           // member object
public:
    Student() { cout << "Student()" << endl; }
    ~Student() { cout << "~Student()" << endl; }
};

class Grad : public Student {
private:
    int *scores;           // heap array
public:
    Grad() : scores(new int[3]) { cout << "Grad()" << endl; }
    ~Grad() { delete [] scores; cout << "~Grad()" << endl; }
};

int main() {
    cout << "-- A --" << endl;
    {
        Grad g;
    }
    cout << "-- B --" << endl;
    Person *p = new Grad;
    delete p;
    cout << "-- C --" << endl;
    return 0;
}

```

Answer

(a) Output

```

-- A --
Person()
Address()
Student()
Grad()
~Grad()
~Student()
~Address()
~Person()
-- B --
Person()
Address()
Student()
Grad()
~Grad()
~Student()
~Address()
~Person()
-- C --

```

Rules:

- **Construction goes from the base down.** For each class, first its base-class part is built, then its member objects, then its own constructor body runs. For a Grad: Person(), then Student's member Address home, then the Student() body, then Grad's member scores, then the Grad() body.
- **Destruction is the exact reverse:** ~Grad, ~Student, ~Address (the member), ~Person.
- In A, g is an automatic (stack) object. It is destroyed at the closing } of its block.

- In B, the object is on the heap. It is destroyed only by `delete p`. Because `~Person` is **virtual**, `delete` through a `Person*` starts at the real type, `Grad`.

(b) Without `virtual`, section B prints:

```
-- B --
Person()
Address()
Student()
Grad()
~Person()
-- C --
```

- `delete p` only calls the destructor of the pointer type, `~Person`.
- `~Grad`, `~Student` and `~Address` never run, so `delete [] scores` never runs. The 3 ints (12 bytes) are a **memory leak**. `valgrind` reports 12 bytes in 1 blocks are definitely lost.
- The C++ standard calls deleting a derived object through a base pointer with a non-virtual destructor **undefined behavior**. Compiled as C++14, `valgrind` also reports `Mismatched new/delete size value: 1: delete used the size of a Person (1 byte), not of a Grad (16 bytes)`.
- `g++ -Wall -Wextra` gives **no warning** for this program.
- This is why the instructor's classes declare `virtual ~Student();`: a class used as a base class, whose objects may be deleted through a base-class pointer, needs a virtual destructor.
- Section A does not change: `g` is a real `Grad`, not reached through a pointer.

Verified: `Dir verify_bank_oop/q04/`.

- `g++ -std=c++11 -Wall -Wextra -pedantic -DVIRT=virtual order.cpp -o ov && ./ov` printed the 19 lines of (a). `valgrind --leak-check=full: All heap blocks were freed -- no leaks are possible, ERROR SUMMARY: 0 errors`.
- `-DVIRT=` (non-virtual) compiled with no warnings. Section B printed `Person() Address() Student() Grad() ~Person()`, and section A was unchanged. `valgrind: 12 bytes in 1 blocks are definitely lost, ERROR SUMMARY: 1 errors from 1 contexts`.
- `-std=c++14 -DVIRT=:valgrind added Mismatched new/delete size value: 1 (2 errors from 2 contexts)`. A size check printed `sizeof Person 1 Grad 16`.
- `clang++` output was identical to `g++` (diff empty).

OBJ-05 · output-prediction

The class below has an `int*` member and **no copy constructor**. (a) What does the program print, and what happens when `main` returns? (b) Write a copy constructor that fixes the problem, and give the new output.

```
#include <iostream>
using namespace std;

class Student {
private:
    int s_id;
    int *data;           // grades, on the heap
    int n;               // number of grades
};
```

```

public:
    Student(int id, int n) : s_id(id), data(new int[n]), n(n) {
        for (int i = 0; i < n; i++) data[i] = 0;
    }
    virtual ~Student() {
        cout << "~Student " << s_id << endl;
        delete [] data;
    }
    void setSID(int id) { s_id = id; }
    void setGrade(int i, int g) { data[i] = g; }
    ostream& print(ostream &out) {
        out << "ID " << s_id << ":";
        for (int i = 0; i < n; i++) out << " " << data[i];
        out << endl;
        return out;
    }
};

int main() {
    Student s1(10, 3);
    s1.setGrade(0, 90);
    Student s2 = s1;           // copy
    s2.setSID(20);
    s2.setGrade(0, 55);
    s1.print(cout);
    s2.print(cout);
    return 0;
}

```

Answer

(a) Output (g++ on Linux)

```

ID 10: 55 0 0
ID 20: 55 0 0
~Student 20
~Student 10
free(): double free detected in tcache 2
Aborted                (exit status 134)

```

Why:

- `Student s2 = s1;` uses the compiler-generated copy constructor. It copies **member by member**: `s_id` and `n` are copied as values, and **only the pointer data is copied**. Both objects now point to the **same heap block**. This is a **shallow copy**, the lecture 9/22 picture.
- `s2.setSID(20)` changes only `s2`'s own `s_id`, so the IDs differ.
- `s2.setGrade(0, 55)` writes into the **shared** block, so `s1` also shows 55. Its 90 is lost.
- Locals are destroyed in reverse order. `~Student 20` runs first and does `delete [] data`, freeing the block. Then `~Student 10` deletes **the same block again**. This is a **double delete (double free)**, which is undefined behavior. Here glibc detects it and aborts the program; on another system the message differs, or the program may crash later.
- `valgrind` shows `Invalid free() / delete / delete[] / realloc()` and `Address 0x... is 0 bytes inside a block of size 12 free'd`. The 12 bytes are 3 ints.
- `g++ -Wall -Wextra` gives **no warning** for this class.

(b) Copy constructor that makes a deep copy

```
// copy constructor - deep copy
// param: other : const Student& - object to copy
Student(const Student &other) : s_id(other.s_id), data(new int[other.n]), n(other.n) {
    for (int i = 0; i < n; i++)
        data[i] = other.data[i];    // copy the VALUES into the new block
}
```

In the class files, declare `Student(const Student&);` in `student.h` and define `Student::Student(const Student &other) ...` in `student.cpp`.

The parameter **must be a reference**. A by-value parameter would itself need a copy, which would call the copy constructor again, forever. g++ rejects it with error: `invalid constructor; you probably meant 'Student (const Student&)'`.

New output

```
ID 10: 90 0 0
ID 20: 55 0 0
~Student 20
~Student 10
```

Each object owns its own block, so each destructor frees a different block. `valgrind` reports 0 errors and no leak. Its heap summary shows 4 `allocs`, 4 `frees`: 2 of these are the program's `new int[3]` blocks, and the other 2 are buffers the C++ library allocates itself.

Verified: Dir `verify_bank_oop/q05/`.

- `g++ -std=c++11 -Wall -Wextra -pedantic shallow.cpp -o shallow` gave no warnings.
`./shallow` printed `ID 10: 55 0 0`, `ID 20: 55 0 0`, `~Student 20`, `~Student 10`, then `free()`:
double free detected in tcache 2. `bash` reported Aborted, `exit=134`.
- `valgrind ./shallow`:

```
Invalid free() / delete / delete[] / realloc()
  by 0x109485: Student::~~Student()
Address 0x4e21080 is 0 bytes inside a block of size 12 free'd
total heap usage: 3 allocs, 4 frees, 77,836 bytes allocated
ERROR SUMMARY: 1 errors from 1 contexts
```

- `-DDEEP` (copy constructor added) printed `ID 10: 90 0 0 / ID 20: 55 0 0 / ~Student 20 / ~Student 10`, `exit=0`. `valgrind`: total heap usage: 4 `allocs`, 4 `frees`, All heap blocks were freed, `ERROR SUMMARY: 0 errors`.
- `-DBYVAL` (`Student(Student other)`): g++ gave error: `invalid constructor; you probably meant 'Student (const Student&)'`. `clang++` gave error: `copy constructor must pass its first argument by reference`.

OBJ-06 · output-prediction

(a) For each line of output, say which function produced it and why: constructor, copy constructor, `operator=` or destructor.

```

#include <iostream>
using namespace std;

class Box {
private:
    int *v;
public:
    Box(int x) : v(new int(x)) { cout << "ctor " << x << endl; }
    Box(const Box &o) : v(new int(*o.v)) { cout << "copy ctor " << *v << endl; }
    Box& operator=(const Box &o) {
        cout << "operator= " << *o.v << endl;
        if (this != &o)
            *v = *o.v;
        return *this;
    }
    virtual ~Box() { cout << "dtor " << *v << endl; delete v; }
    void set(int x) { *v = x; }
};

void byValue(Box b) { b.set(8); cout << "in byValue" << endl; }
void byRef(Box &b) { b.set(9); cout << "in byRef" << endl; }

int main() {
    Box a(1);           // L1
    Box b = a;           // L2
    b.set(2);
    Box c(3);           // L3
    c = a;               // L4
    byValue(a);          // L5
    byRef(c);            // L6
    Box *p = new Box(b); // L7
    delete p;            // L8
    cout << "end of main" << endl;
    return 0;
}

```

(b) The `Student` class from OBJ-05 now **has** the deep copy constructor but **no** `operator=`. What happens with the `main` below?

```

int main() {
    Student s1(10, 3);
    s1.setGrade(0, 90);
    Student s2(20, 2);
    s2 = s1;           // assignment
    s2.setGrade(0, 55);
    s1.print(cout);
    s2.print(cout);
    s1 = s1;           // self assignment
    s1.print(cout);
    return 0;
}

```

(c) Write `operator=` for `Student` so that it makes a deep copy.

Answer

(a) Output

```

ctor 1
copy ctor 1
ctor 3

```



```

operator= 1
copy ctor 1
in byValue
dtor 8
in byRef
copy ctor 2
dtor 2
end of main
dtor 9
dtor 2
dtor 1

```

output	produced by	why
ctor 1	constructor (L1)	Box a(1)
copy ctor 1	copy constructor (L2)	Box b = a; creates a NEW object from a. This = is initialization, not operator=
ctor 3	constructor (L3)	Box c(3)
operator= 1	operator= (L4)	c = a; where c already exists
copy ctor 1	copy constructor (L5)	the by-value parameter is a new copy of a
in byValue	body of byValue	the parameter copy is set to 8
dtor 8	destructor (L5)	the parameter copy dies when byValue returns. a is unchanged
in byRef	body of byRef (L6)	by reference: no copy and no destructor. c itself becomes 9
copy ctor 2	copy constructor (L7)	new Box(b), and b holds 2 because of b.set(2)
dtor 2	destructor (L8)	delete p
end of main	main	
dtor 9, dtor 2, dtor 1	destructors	locals are destroyed in reverse order of creation: c (9), b (2), a (1)

Rule:

- A copy **constructor** runs whenever a **new** object is created from an existing one: `Box b = a;`, `Box b(a);`, passing by value, `new Box(a)`.
- **operator=** runs when you assign to an object that **already exists**.
- Passing by reference makes no copy.

(b) Only the copy constructor was written, so `s2 = s1` uses the **compiler-generated operator=**, which is a shallow copy.

- With **-Wextra** (not with **-Wall** alone), g++ warns: `implicitly-declared 'Student& Student::operator=(const Student&)' is deprecated [-Wdeprecated-copy]`.
- Output:

```

ID 10: 55 0 0
ID 10: 55 0 0

```

```
ID 10: 55 0 0
~Student 10
~Student 10
free(): double free detected in tcache 2
Aborted
```

- After the assignment, s1 and s2 share one block, so setting a grade through s2 also changes s1. s_id was copied too, so both print ID 10.
- s2's original 2-int block is never freed: a **memory leak** of 8 bytes.
- Both destructors delete the same block: a **double free**.
- valgrind: Invalid free() plus 8 bytes in 1 blocks are definitely lost.

(c)

```
// assignment operator - deep copy
// param: other : const Student& - object to copy from
// return: Student& - this object (allows a = b = c)
Student& Student::operator=(const Student &other) {
    if (this != &other) { // s1 = s1 must not destroy its own data
        int *temp = new int[other.n]; // new block
        for (int i = 0; i < other.n; i++)
            temp[i] = other.data[i]; // copy the values
        delete [] data; // free the OLD block (else memory leak)
        data = temp;
        n = other.n;
        s_id = other.s_id;
    }
    return *this;
}
```

(Declare Student& operator=(const Student&); in the class.) Output after the fix:

```
ID 10: 90 0 0
ID 10: 55 0 0
ID 10: 90 0 0
~Student 10
~Student 10
```

valgrind reports 0 errors and no leak.

Rule of three: if a class needs a destructor because it owns heap memory, it also needs a copy constructor and an operator=.

Verified: Dir verify_bank_oop/q06/.

- g++ -std=c++11 -Wall -Wextra -pedantic box.cpp -o box && ./box gave no warnings and printed exactly the 14 lines above. clang++ output was identical. valgrind: All heap blocks were freed, ERROR SUMMARY: 0 errors.
- (b) g++ -std=c++11 -Wall assign.cpp gave 0 warnings. g++ -std=c++11 -Wall -Wextra -pedantic assign.cpp gave assign.cpp:56:10: warning: implicitly-declared 'Student& Student::operator=(const Student&)' is deprecated [-Wdeprecated-copy], plus the same warning at line 60. The run printed the three ID 10: 55 0 0 lines, two ~Student 10 lines and free(): double free detected in tcache 2, exit=134. valgrind: Invalid free() / delete /

delete[] / realloc(), 8 bytes in 1 blocks are definitely lost, ERROR SUMMARY: 2 errors from 2 contexts.

- (c) -DASSIGN printed ID 10: 90 0 0 / ID 10: 55 0 0 / ID 10: 90 0 0 / ~Student 10 / ~Student 10, exit=0.valgrind: total heap usage: 5 allocs, 5 frees, All heap blocks were freed, ERROR SUMMARY: 0 errors.

OBJ-07 · find-the-bug

Which lines of `main` do **not** compile? Explain using the difference between `struct` and `class`, and fix the types so that all of `main` compiles.

```
#include <iostream>
using namespace std;

struct Point {
    int x, y;
    void print() { cout << "(" << x << "," << y << ")" << endl; }
};

class PointC {
    int x, y;
    void print() { cout << "(" << x << "," << y << ")" << endl; }
};

struct Point3 : Point {
    int z;
};

class Point3C : Point {
    int z;
};

int main() {
    Point p = {1, 2};           // line 23
    p.x = 5;                   // line 24
    p.print();                 // line 25
    PointC c;                  // line 26
    c.x = 5;                   // line 27
    c.print();                 // line 28
    Point3 q;                  // line 29
    q.x = 7;                   // line 30
    q.z = 8;                   // line 31
    Point3C r;                 // line 32
    r.x = 7;                   // line 33
    r.z = 8;                   // line 34
    return 0;
}
```

Answer

In C++ a `struct` and a `class` differ only in their **defaults**:

- **Member access:** members of a `struct` are **public** by default. Members of a `class` are **private** by default.
- **Inheritance:** `struct D : B` means `: public B`. `class D : B` means `: private B`.

line	result
23-25	OK. Point's members are public, so brace initialization {1, 2} also works. p.print() prints (5,2)
26 PointC c;	OK. Creating the object is allowed because the implicit default constructor is public
27 c.x = 5;	error: 'int PointC::x' is private within this context (class, so private by default)
28 c.print();	error: 'void PointC::print()' is private within this context (methods are private by default too)
29-31	OK. struct Point3 : Point is public inheritance, so x stays public in Point3, and z is public
32	OK
33 r.x = 7;	error: 'int Point::x' is inaccessible within this context. class Point3C : Point is private inheritance, so Point's public members become private inside Point3C
34 r.z = 8;	error: 'int Point3C::z' is private within this context

Related: PointC c = {1, 2}; also fails, with could not convert '{1, 2}' from '<brace-enclosed initializer list>' to 'PointC'. Brace (aggregate) initialization needs public data.

Fix:

```
class PointC {
public:
    int x, y;
    PointC() : x(0), y(0) {} // constructor: y is never left uninitialized
    void print() { cout << "(" << x << "," << y << ")" << endl; }
};

class Point3C : public Point {
public:
    int z;
};
```

Now all of main compiles. Lines 25 and 28 print (5,2) and (5,0).

Why the constructor: if you only add public:, main still never sets c.y, so line 28 prints an **uninitialized** value. One run printed (5,1819242352), and valgrind reported Conditional jump or move depends on uninitialised value(s).

This is the lecture's convention: a struct is a "primitive object" with public visibility, and a class is a "full object" with information hiding and a constructor.

Verified: Dir verify_bank_oop/q07/.

- g++ -std=c++11 -Wall -Wextra -c sc.cpp (the question's code):

```
sc.cpp:27:7: error: 'int PointC::x' is private within this context
sc.cpp:28:12: error: 'void PointC::print()' is private within this context
sc.cpp:33:7: error: 'int Point::x' is inaccessible within this context
sc.cpp:34:7: error: 'int Point3C::z' is private within this context
```

There were no errors on lines 23-26 or 29-32.

- `agg.cpp` (`PointC c = {1, 2};`): error: could not convert '{1, 2}' from '<brace-enclosed initializer list>' to 'PointC'.
- Fix with only public: added (`sc_fixed.cpp`): it printed (5,2) then (5,1819242352). `valgrind`: Conditional jump or move depends on uninitialised value(s), Use of uninitialised value of size 8. `g++ -O2 -Wall -Wextra`: warning: 'c.PointC::y' may be used uninitialized [-Wmaybe-uninitialized].
- Fix with the constructor (`sc_fixed2.cpp`): no errors (only -Wunused-but-set-variable for q and r). It printed (5,2) / (5,0). `valgrind`: ERROR SUMMARY: 0 errors.

OBJ-08 · output-prediction

(a) What does this program print? Explain the role of `this` on each marked line. (b) Fix constructor (1) in two different ways.

```
#include <iostream>
#include <string>
using namespace std;

class Counter {
private:
    string name;
    int count;
public:
    Counter(string name) : count(0) { name = name; }           // (1)
    void setName(string name) { this->name = name; }           // (2)
    Counter& inc()      { count++; return *this; }               // (3)
    Counter incCopy() { count++; return *this; }                 // (4)
    bool same(const Counter &o) { return this == &o; }          // (5)
    ostream& print(ostream &out) {
        out << "[" << name << "]" << count << endl;
        return out;
    }
};

int main() {
    Counter a("a");
    a.print(cout); // A
    a.setName("alpha");
    a.inc().inc().inc();
    a.print(cout); // B
    Counter b("b");
    b.setName("beta");
    b.incCopy().incCopy().incCopy();
    b.print(cout); // C
    cout << a.same(a) << " " << a.same(b) << endl; // D
    a.inc().print(cout); // E
    return 0;
}
```

(c) Why does an assignment operator start with `if (this != &other)`? What happens if it is missing and the operator frees its old data first?

Answer

this is a pointer to the object the method was called on. For `s.print()`, `this == &s`.

(a) Output

```
[] 0
[alpha] 3
[beta] 1
1 0
[alpha] 4
```

- **A:** in constructor (1), the parameter `name` **hides** the member, so `name = name;` assigns the parameter to itself. The member keeps its default value, an empty string, which prints as `[]`. `g++ -Wall -Wextra` gives **no warning**; `g++ -Wshadow` warns declaration of 'name' shadows a member of 'Counter'. `clang++` warns explicitly assigning value of variable of type 'string' ... to itself; did you mean to assign to member 'name'? `[-Wself-assign-overloaded]`.
- **B:** (2) `this->name` is the member, so the name becomes alpha. (3) `inc()` returns `*this` **by reference**, so each `.inc()` in the chain acts on `a` itself: count is 3.
- **C:** (4) `incCopy()` returns `*this` **by value**, which is a copy. The first call increments `b`. The second and third calls increment temporary copies. `b.count` is 1.
- **D:** (5) `this == &o` compares **addresses**. The same object gives 1 (true). A different object gives 0 (false).
- **E:** `inc()` returns `Counter&` and `print` returns `ostream&`, so calls can be chained. The count becomes 4.

(b)

```
Counter(string name) : count(0) { this->name = name; } // use this->
Counter(string name) : name(name), count(0) {} // or the initializer list:
member(parameter)
```

(A third way is to rename the parameter, for example `n`.) Both fixes print `[a] 0` on line A.

(c) For self-assignment `v = v;`, other is `*this`: `&other == this`, and `other.arr` is the same pointer as `arr`. An operator that frees first destroys the object's own data:

```
IntVector& IntVector::operator=(const IntVector &other) {
    delete [] arr; // also frees other.arr (same block!)
    arr = new int[other.max_elements]; // other.arr now names this NEW, uninitialized block
    max_elements = other.max_elements;
    curr_element = other.curr_element;
    for (int i = 0; i < curr_element; i++)
        arr[i] = other.arr[i]; // copies garbage onto itself
    return *this;
}
```

A vector holding 1 2 3 printed garbage after `v = v;`. Three runs gave 1453713362 5 0, 1498050810 5 0 and 1544704884 5 0, and `valgrind` reported Conditional jump or move depends on uninitialised value(s). Adding `if (this == &other) return *this;` at the top keeps 1 2 3. Allocating and copying into a temp before the delete, as in OBJ-06, also fixes it.

Verified: Dir verify_bank_oop/q08/.

- `g++ -std=c++11 -Wall -Wextra -pedantic thisp.cpp -o thisp && ./thisp` gave no warnings and printed:

```
[ ] 0
[alpha] 3
[beta] 1
1 0
[alpha] 4
```

- `clang++` gave the same output plus `thisp.cpp:10:48: warning: explicitly assigning value of variable of type 'string' (aka 'basic_string<char>') to itself; did you mean to assign to member 'name'? [-Wself-assign-overloaded]`.
- `g++ -Wshadow: thisp.cpp:10:24: warning: declaration of 'name' shadows a member of 'Counter' [-Wshadow]`.
- Both constructor fixes (`fix1.cpp`, `fix2.cpp`) printed `[a] 0` as the first line.
- `selfassign.cpp` (the `operator=` above): 3 runs printed `1453713362 5 0`, `1498050810 5 0`, `1544704884 5 0`. `valgrind: Conditional jump or move depends on uninitialised value(s), Use of uninitialised value of size 8, ERROR SUMMARY: 12 errors from 4 contexts. With the this == &other check (-DCHECK): printed 1 2 then 1 2 3 twice... precisely 1 2 3 / 1 2 3, valgrind 0 errors.`

OBJ-09 · short-answer

(a) For each numbered line, say whether it is a **declaration only** or a **definition**, and whether it belongs in a `.h` or a `.cpp` file.

```
class Book; // 1
class Book { // 2
    private:
        std::string title;
        int pages;
    public:
        Book(std::string, int); // 3
        int getPages(); // 4
        int twice() { return 2 * pages; } // 5
};
Book::Book(std::string t, int p) : title(t), pages(p) {} // 6
int Book::getPages() { return pages; } // 7
extern int total; // 8
int total = 0; // 9
int maxOf(int, int); // 10
int maxOf(int a, int b) { return a > b ? a : b; } // 11
Book b("Dune", 412); // 12
```

(b) What error do you get, and from which stage (compiler or linker), in each case?

- i. `book.h` holds the class definition (line 2) and `book.cpp` holds lines 6 and 7. `main.cpp` includes `book.h` and does `Book b("Dune", 412); cout << b.getPages() << " " << b.twice() << endl;`. You compile with `g++ main.cpp -o book`.
- ii. A file sees only line 1 (it does not include `book.h`) and does `Book b("Dune", 412);`.

- iii. `book.h` has no include guard, and `main.cpp` includes it twice (directly and through another header).
- iv. A header (with an include guard) defines `void Counter::inc() { count++; }` **outside** the class body, and two `.cpp` files include that header.

Answer

A **declaration** introduces a name and its type. It may appear many times. A **definition** gives the body or allocates the storage.

One-definition rule: a non-inline function or a variable must be defined **exactly once in the whole program**. A class definition, and a function defined inside it (inline), may appear once in **each .cpp file** that needs it. That is why they can live in a `.h` file that several `.cpp` files include, with an include guard.

(a)

line	kind	file
1 <code>class Book;</code>	declaration only (forward declaration): "Book is a class". It allows <code>Book*</code> and <code>Book&</code> , not <code>Book b</code>	<code>.h</code>
2 <code>class Book { ... };</code>	class definition, which is the instructor's "class declaration" in <code>student.h</code> . It fixes the members and size of the type, but lines 3 and 4 inside it are only method declarations (prototypes)	<code>.h</code>
3, 4	member function declarations	<code>.h</code>
5	declaration and definition: the body is inside the class, so it is implicitly inline and allowed in a <code>.h</code>	<code>.h</code>
6, 7	definitions of members, using the <code>Book::</code> scope	<code>.cpp</code> (<code>book.cpp</code>)
8 <code>extern int total;</code>	declaration only: no storage	<code>.h</code>
9 <code>int total = 0;</code>	definition: allocates the global (static allocation, global space)	<code>.cpp</code>
10	declaration (prototype)	<code>.h</code>
11	definition (has a body)	<code>.cpp</code>
12	definition of an object (instance): allocates it and calls the constructor	<code>.cpp</code>

The listing compiles and runs. With a `main` that prints `maxOf(b.getPages(), b.twice())`, the output is `824`.

(b)

- i. **Linker** error. `main.cpp` saw only the declarations in `book.h`. The bodies of lines 6 and 7 are in `book.cpp`, which was not compiled or linked:

```
undefined reference to `Book::Book(std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> >, int)'
undefined reference to `Book::getPages()'
collect2: error: ld returned 1 exit status
```


There is **no** error for `twice()` : its body is inside the class (line 5), so `main.cpp` already has it. Fix: `g++ main.cpp book.cpp -o book`, which prints 412 824.

- ii. **Compiler** error: variable 'Book b' has initializer but incomplete type. (For `Book b`; without arguments, `g++` says aggregate 'Book b' has incomplete type and cannot be defined.) To create an object, the compiler needs the full class definition, because it must know the object's size and its constructor. A pointer `Book *ptr`; compiles.
- iii. **Compiler** error: redefinition of 'class Book'. Fix it with `#ifndef BOOK_H_ / #define BOOK_H_ / ... / #endif`.
- iv. **Linker** error. An include guard only protects one .cpp file, not two, so the function body is compiled into both .o files:

```
multiple definition of `Counter::inc()'; ...a.cpp:(.text+0x0): first defined here
```

Fix it by moving the definition to `counter.cpp`, writing it inside the class body, or marking it `inline`.

Verified: `Dir verify_bank_oop/q09/`.

- `g++ -std=c++11 -Wall -Wextra -pedantic listing.cpp -o listing && ./listing` gave no warnings and printed 824.
- (i) `g++ -std=c++11 -Wall -Wextra main.cpp -o book`:

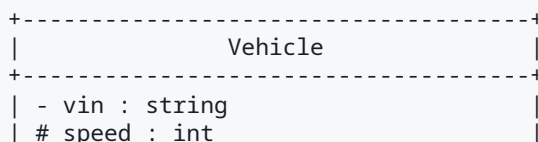
```
main.cpp:(.text+0x54): undefined reference to `Book::Book(std::__cxx11::basic_string<char,
std::char_traits<char>, std::allocator<char> >, int)'
main.cpp:(.text+0x79): undefined reference to `Book::getPages()'
collect2: error: ld returned 1 exit status
```

(no error for `twice`). `g++ main.cpp book.cpp -o book && ./book` printed 412 824.

- (ii) `fwd.cpp` (`class Book`; then `Book *ptr = nullptr`; and `Book b("Dune", 412);`): the only error was `fwd.cpp:4:11: error: variable 'Book b' has initializer but incomplete type`, so the pointer line compiled. `fwd2.cpp` (`Book b`;): `fwd2.cpp:3:10: error: aggregate 'Book b' has incomplete type and cannot be defined`.
- (iii) `book.h:2:7: error: redefinition of 'class Book'`.
- (iv) `main.cpp:(.text+0x0): multiple definition of Counter::inc()'; /tmp/cc...o:a.cpp:(.text+0x0): first defined here`. With `inline` it linked and printed 1`.
- Only extern `int total`; and the `maxOf` prototype with no definitions: undefined reference to `total` and undefined reference to `maxOf(int, int)`'.

OBJ-10 · short-answer

Read the UML diagram below.





(a) What do `-`, `+` and `#` mean? (b) Classify the three relationships (Car-Vehicle, Car-Engine, Driver-Car) as is-a, has-a or association, and say how each one appears in C++. (c) Write the `Car` class declaration and its constructor definition. Why must `Vehicle(vin)` and `engine(hp)` be in the initializer list? (d) Inside `Car::print`, may you write `out << vin;`? What about `out << speed;`? May `main` write `c.speed = 100;`?

Answer

(a)

- **private**: only the class's own methods can use it.
- + **public**: any code can use it.
- # **protected**: the class and its **derived** classes can use it, but outside code (for example `main`) cannot.

This is information hiding.

(b)

- Car to Vehicle, hollow triangle: inheritance, "is-a"**. A Car is a Vehicle. In C++: `class Car : public Vehicle`.
- Car with a filled diamond to Engine: composition, "has-a"**. A Car has an Engine. The engine is part of the car and lives and dies with it. In C++: a member object, `Engine engine;`
- Driver to Car, plain line labeled drives: association**. The Driver uses a Car but does not own it, and both exist independently. This is like the lecture's Student —mentor— Faculty, which is "not inheritance". In C++: a pointer member, `Car *car;`, which Driver does **not** delete.

(c)

```

// car.h
#ifndef CAR_H_
#define CAR_H_
#include <iostream>
#include <string>
#include "vehicle.h"
#include "engine.h"

class Car : public Vehicle {
private:
    Engine engine;
    int doors;
public:
    Car(std::string, int, int);
    void accelerate(int);
    std::ostream& print(std::ostream&);
};
#endif

// car.cpp
#include "car.h"

Car::Car(std::string vin, int hp, int doors) : Vehicle(vin), engine(hp), doors(doors) {}

void Car::accelerate(int dv) { speed += dv; } // speed is protected: OK here

std::ostream& Car::print(std::ostream &out) {
    out << "Car " << getVin() << " speed " << speed
        << " hp " << engine.getHp() << " doors " << doors << std::endl;
    return out;
}

```

The base part and the member objects are constructed **before** the constructor body runs. Vehicle and Engine have **no default constructor**, because each declares only a constructor with a parameter. The arguments must therefore be passed in the initializer list:

- without `Vehicle(vin)`: error: no matching function for call to 'Vehicle::Vehicle()'
- without `engine(hp)`: error: no matching function for call to 'Engine::Engine()'

(d)

- `out << vin;` gives error: 'std::string Vehicle::vin' is private within this context. Private members are inherited (they exist inside every Car) but are **not accessible** in the derived class. Use `getVin()`.
- `out << speed;` is OK, because # protected members are accessible in derived classes.
- In `main`, `c.speed = 100;` gives error: 'int Vehicle::speed' is protected within this context.

Test program. The UML does not show `virtual`. For a `Vehicle*` to reach Car's print, you must declare `print` (and the destructor) **virtual** in Vehicle:

```

class Vehicle {
private:
    std::string vin;
protected:
    int speed;
public:
    Vehicle(std::string vin) : vin(vin), speed(0) {}

```

```

    virtual ~Vehicle() {}
    std::string getVin() { return vin; }
    virtual std::ostream& print(std::ostream &out) {
        out << "Vehicle " << vin << " speed " << speed << std::endl;
        return out;
    }
};

class Engine {
private:
    int hp;
public:
    Engine(int hp) : hp(hp) {}
    int getHp() { return hp; }
};

class Driver {
private:
    std::string name;
    Car *car; // association: uses a Car, does not own it
public:
    Driver(std::string name) : name(name), car(nullptr) {}
    void drive(Car *c) { car = c; }
    std::ostream& print(std::ostream &out) {
        out << name << " drives ";
        if (car != nullptr) car->print(out); else out << "nothing" << std::endl;
        return out;
    }
};

```

The test was `Car c("1HGCM82633A", 150, 4); c.accelerate(30); Driver d("Ana"); d.drive(&c); d.print(cout); Vehicle *v = &c; v->print(cout);`. It printed:

```

Ana drives Car 1HGCM82633A speed 30 hp 150 doors 4
Car 1HGCM82633A speed 30 hp 150 doors 4

```

The second line is the Car version even through a `Vehicle*`, because `print` is virtual. If `virtual` is removed from Vehicle's `print`, that line becomes `Vehicle 1HGCM82633A speed 30`.

Verified: Dir verify_bank_oop/q10/ (uml.cpp contains the code above).

- `g++ -std=c++11 -Wall -Wextra -pedantic uml.cpp -o uml && ./uml` gave no warnings and printed the two lines above. `valgrind: ERROR SUMMARY: 0 errors`.
- -DBAD1 (vin in `Car::print`): `uml.cpp:50:12: error: 'std::string Vehicle::vin' is private within this context`.
- -DBAD2 (`c.speed = 100` in main): `uml.cpp:75:7: error: 'int Vehicle::speed' is protected within this context`.
- -DNOVEH (no `Vehicle(vin)`): `error: no matching function for call to 'Vehicle::Vehicle()'`.
- -DNOENG (no `engine(hp)`): `error: no matching function for call to 'Engine::Engine()'`.
- `uml_nv.cpp` (virtual removed from `Vehicle::print`): its second line was `Vehicle 1HGCM82633A speed 30`.

A self-sizing array (vector) of ints is declared as follows:

```
class IntVector {
private:
    int *arr;           // data array on the heap
    int max_elements;   // capacity
    int curr_element;   // number of elements in use
    void resize();      // grow: new max = max + factor*max, factor = 1
public:
    IntVector(int);     // initial capacity
    virtual ~IntVector();
    void addAtEnd(const int&);
    int removeAt(int);   // 0 = ok, -1 = bad index
    int size();
    int capacity();
    std::ostream& print(std::ostream&);
};
```

(a) Write `resize`, `addAtEnd` and `removeAt`. `removeAt` removes the element at the given index and shifts the remaining elements left. (b) Show size, capacity and contents after each step of: `IntVector v(2);` then `addAtEnd(5), (8), (1), (9), (3)`, then `removeAt(1), removeAt(3), removeAt(7)`. (c) What is the running time of `addAtEnd` and `removeAt`?

Answer

(a)

```
// resize - allocate a bigger array, copy, delete the old one
void IntVector::resize() {
    int newMax = max_elements + 1 * max_elements; // factor 1 -> double
    int *temp = new int[newMax];
    for (int i = 0; i < curr_element; i++)
        temp[i] = arr[i]; // copy the data
    delete [] arr;        // free the old array (else memory leak)
    arr = temp;
    max_elements = newMax;
}

// add at the end
// param: e : const int& - element to add
void IntVector::addAtEnd(const int &e) {
    if (curr_element == max_elements) // check if full
        resize(); // true: resize
    arr[curr_element] = e; // add to next location
    curr_element++;
}

// remove the element at index loc, shift the rest left
// param: loc : int - index to remove
// return: int - 0 ok, -1 bad index
int IntVector::removeAt(int loc) {
    if (loc < 0 || loc >= curr_element)
        return -1;
    for (int i = loc; i < curr_element - 1; i++)
        arr[i] = arr[i + 1]; // shift left
    curr_element--;
    return 0;
}
```

The rest of the class, for completeness:

```
// param: m : int - initial capacity
IntVector::IntVector(int m) : arr(new int[m]), max_elements(m), curr_element(0) {}
IntVector::~IntVector() { delete [] arr; }
int IntVector::size() { return curr_element; }
int IntVector::capacity() { return max_elements; }
```

Careful: members are initialized in the order they are **declared in the class** (`arr` first), not in the order of the initializer list. Writing `arr(new int[max_elements])` would use `max_elements` before it is set. g++ warns `'*this.IntVector::max_elements' is used uninitialized, and valgrind reports errors. Use the parameter m.`

(b) Trace

```
start          -> size 0 cap 2 :
add 5          -> size 1 cap 2 : 5
add 8          -> size 2 cap 2 : 5 8
add 1          -> size 3 cap 4 : 5 8 1      (full: resize 2 -> 4, copy 2)
add 9          -> size 4 cap 4 : 5 8 1 9
add 3          -> size 5 cap 8 : 5 8 1 9 3  (full: resize 4 -> 8, copy 4)
removeAt(1) = 0 -> size 4 cap 8 : 5 1 9 3  (1, 9, 3 shifted left)
removeAt(3) = 0 -> size 3 cap 8 : 5 1 9    (last element: no shifting)
removeAt(7) = -1 -> size 3 cap 8 : 5 1 9   (bad index, nothing changes)
```

Removing elements does not reduce the capacity. Shrinking would need a separate compress step.

(c)

- `addAtEnd` is $O(1)$ when the array is not full. It is $O(N)$ when it must resize, because N elements are copied. With multiplicative growth the **amortized** cost is $O(1)$ per add (see OBJ-12).
- `removeAt(i)` does $N-1-i$ shifts: $O(N)$ in the worst case (index 0) and $O(1)$ for the last element.
- Access by index is $O(1)$.

Verified: Dir verify_bank_oop/q11/ (intvector.h, intvector.cpp with exactly the code above, main.cpp driver).

- `g++ -std=c++11 -Wall -Wextra -pedantic main.cpp intvector.cpp -o vec && ./vec` gave no warnings and printed:

```
start -> size 0 cap 2 :
add 5 -> size 1 cap 2 : 5
add 8 -> size 2 cap 2 : 5 8
add 1 -> size 3 cap 4 : 5 8 1
add 9 -> size 4 cap 4 : 5 8 1 9
add 3 -> size 5 cap 8 : 5 8 1 9 3
removeAt(1) = 0 -> size 4 cap 8 : 5 1 9 3
removeAt(3) = 0 -> size 3 cap 8 : 5 1 9
removeAt(7) = -1 -> size 3 cap 8 : 5 1 9
```

- `valgrind --leak-check=full ./vec`: All heap blocks were freed -- no leaks are possible, ERROR SUMMARY: 0 errors.
- `-DBADCTOR (arr(new int[max_elements]))`: intvector.cpp:4:43: warning: `'*this.IntVector::max_elements' is used uninitialized [-Wuninitialized]`. valgrind:

Conditional jump or move depends on uninitialised value(s), ERROR SUMMARY: 2 errors from 2 contexts.

OBJ-12 · trace

A vector starts **empty with capacity 4**, and `addAtEnd` is called **20 times**. When the vector is full, `resize` makes a new array and copies every element. For each growth policy from the lecture, give the final capacity, the number of resizes, and the total number of element copies:

1. `max_elements + 1`
2. `max_elements + 100`
3. `max_elements + factor * max_elements` with factor = 1 (doubling)

Then explain the **amortized** cost per `addAtEnd` for each policy as N grows. Why is doubling preferred, and what is its disadvantage?

Answer

20 adds starting from capacity 4

policy	resizes happen at add #	final capacity	resizes	element copies
+1	5, 6, ..., 20 (every add once full)	20	16	$4+5+\dots+19 = \mathbf{184}$
+100	5 (4 to 104)	104	1	4
x2	5 (4 to 8), 9 (8 to 16), 17 (16 to 32)	32	3	$4+8+16 = \mathbf{28}$

Amortized cost (total copies for N adds divided by N; start capacity 1, counted by a program):

N	+1	+100	x2
1,000	499,500 (499.5 per add)	4,510 (4.51)	1,023 (1.02)
10,000	49,995,000 (4999.5)	495,100 (49.5)	16,383 (1.64)
1,000,000	499,999,500,000 (500,000)	4,999,510,000 (4999.5)	1,048,575 (1.05)

- **+1:** copies = $1+2+\dots+(N-1) = N(N-1)/2$. That is $O(N^2)$ in total, so each add costs **$O(N)$ amortized**. Every add after the first fill triggers a resize.
- **+100 (any constant c):** about $N^2/(2c)$ copies. This is 100 times fewer than +1 but **still $O(N^2)$** in total, $O(N)$ per add. The constant is better, but the growth rate is the same.
- **x2 (any factor > 1):** copies = $1+2+4+\dots+(\text{last capacity}) < 2N$. That is $O(N)$ in total, so each add is **$O(1)$ amortized**. An occasional expensive resize is paid for by many cheap adds.

Measured CPU time (`getCPUTime` from the instructor's support.h, real heap array, start capacity 1) for N = 25k, 50k and 100k:

- +1: 0.67 s, 2.70 s, 10.7 s. Doubling N makes it about 4 times slower: **quadratic**.
- +100: 0.0061 s, 0.024 s, 0.093 s. Also about 4 times per doubling of N: **quadratic**, with a smaller constant.

- x2: 0.00028 s, 0.00037 s, 0.00078 s. These times are so small they are noisy, but they grow roughly in proportion to N: **linear**.

Times depend on the machine; the ratios are the point.

Disadvantage of doubling: wasted memory. Right after a resize, about half of the capacity is unused (here 32 slots hold 20 elements); the capacity stays below 2N. With +100, at most 99 slots are unused.

Doubling needs a starting capacity of at least 1, since $2 \times 0 = 0$. `std::vector` in g++'s library (libstdc++) also doubles (see OBJ-13).

Verified: Dir verify_bank_oop/q12/.

- growth.cpp (a simulation that counts resizes and copies), built with `g++ -std=c++11 -Wall -Wextra -pedantic`, printed:

```
policy +1, start cap 4, 20 adds
  final size 20, capacity 20, resizes 16, copies 184
policy +100, start cap 4, 20 adds
  add #5: resize 4 -> 104 (copy 4)
  final size 20, capacity 104, resizes 1, copies 4
policy x2, start cap 4, 20 adds
  add #5: resize 4 -> 8 (copy 4)
  add #9: resize 8 -> 16 (copy 8)
  add #17: resize 16 -> 32 (copy 16)
  final size 20, capacity 32, resizes 3, copies 28
N=1000  +1: 499500 (499.5)  +100: 4510 (4.51)  x2: 1023 (1.023)
N=10000 +1: 49995000 (4999.5) +100: 495100 (49.51) x2: 16383 (1.6383)
N=100000 +1: 4999950000 (500000) +100: 4999510000 (4999.51) x2: 1048575 (1.04858)
+1 resize at adds: 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

- timing.cpp, a real heap array timed with the instructor's support.cpp (`g++ -std=c++11 -Wall -Wextra timing.cpp support.cpp -o timing`), printed:

```
N=25000  +1: 0.667211 s  +100: 0.006068 s  x2: 0.000277 s
N=50000  +1: 2.69914 s  +100: 0.023554 s  x2: 0.000367 s
N=100000 +1: 10.6945 s  +100: 0.093277 s  x2: 0.000775 s
```

OBJ-13 · output-prediction

What does this program print (g++ with its standard library)? Which outputs are guaranteed by C++, and which depend on the library? What is the difference between `v[i]` and `v.at(i)`?

```
#include <iostream>
#include <vector>
#include <stdexcept>
using namespace std;

int main() {
    vector<int> v;
    cout << v.size() << " " << v.capacity() << endl;           // A
    for (int i = 1; i <= 5; i++) {
        v.push_back(i * 10);
        cout << v.size() << " " << v.capacity() << endl;       // B (5 lines)
    }
}
```



```

}
v[0] = 7;
cout << v[2] << " " << v.at(4) << " " << v.back() << endl; // C
v.pop_back();
cout << v.size() << " " << v.capacity() << endl;           // D
try {
    cout << v.at(10) << endl;                                // E
} catch (const out_of_range &e) {
    cout << "out_of_range: " << e.what() << endl;
}
vector<int> w = v;                                           // F
w[0] = 99;
cout << v[0] << " " << w[0] << " " << w.size() << " " << w.capacity() << endl;
vector<int> r;
r.reserve(100);
cout << r.size() << " " << r.capacity() << endl;           // G
return 0;
}

```

Answer

Output (g++ 13 / libstdc++)

```

0 0
1 1
2 2
3 4
4 4
5 8
30 50 50
4 8
out_of_range: vector::_M_range_check: __n (which is 10) >= this->size() (which is 4)
7 99 4 4
0 100

```

- **A/B:** `size()` is the number of elements. `capacity()` is the number of slots allocated. When full, libstdc++ **doubles** the capacity (1, 2, 4, 8), the same as the lecture's `max + 1*max` policy.
 - The standard only guarantees `capacity() >= size()` and **amortized O(1)** `push_back`.
 - The exact capacities (including 0 for an empty vector) depend on the library; other libraries may use a different growth factor.
- **C:** `v[2]` is 30, `v.at(4)` is 50, and `back()` is the last element, 50. `v[0] = 7` replaced the 10.
- **D:** `pop_back()` reduces the size to 4 and leaves the **capacity at 8**. The vector does not shrink by itself.
- **E:** `at()` **checks the index** and throws `std::out_of_range`. The `what()` text shown is libstdc++'s wording; only the exception type is guaranteed.
- **F:** copying a `vector` makes a **deep copy**. Its copy constructor allocates a new array, so changing `w[0]` does not change `v[0]` (7 and 99). In libstdc++ the copy's capacity is 4, the size of the data copied, not 8. The standard only guarantees at least the size.
- **G:** `reserve(100)` allocates room for at least 100 elements (exactly 100 here). The size is still 0.

`v[i]` vs `v.at(i)`:

- `operator[]` does **no bounds check**. It is fast, and an out-of-range index is undefined behavior: it reads or writes whatever memory is there, with no exception.
- `at()` checks the index and throws `std::out_of_range`.

- Tested with `vector<int> v = {7, 20, 30, 40};` (size 4, capacity 4): `v[10]` ran silently. `valgrind` reported `Invalid read of size 4 ... 24 bytes after a block of size 16.`

Verified: Dir `verify_bank_oop/q13/`.

- `g++ -std=c++11 -Wall -Wextra -pedantic stdvec.cpp -o stdvec && ./stdvec` gave no warnings and printed exactly the 11 lines above. `clang++ 18` (same `libstdc++`) gave identical output on all 11 lines (diff empty). `valgrind`: All heap blocks were freed, ERROR SUMMARY: 0 errors.
- `bracket.cpp` (`vector<int> v = {7, 20, 30, 40}; int x = v[10];`) printed `v[10]` read, no exception, `exit 0`. `valgrind ./bracket`: `Invalid read of size 4, Address 0x4e210a8 is 24 bytes after a block of size 16 in arena "client", ERROR SUMMARY: 1 errors from 1 contexts.`

OBJ-14 · find-the-bug

Each version of `IntVector::resize()` below has one bug. Name it, say what happens when the program runs, and fix it.

The program is `IntVector v(2);` followed by `addAtEnd(10), (20), ..., (60)` and then `print`. The constructor does `arr(new int[m]), max_elements(m), curr_element(0)`, and the destructor does `delete [] arr;`.

```
void addAtEnd(int e) {
    if (curr_element == max_elements)
        resize();
    arr[curr_element] = e;
    curr_element++;
}
```

Version 1

```
void resize() {
    int *temp = new int[max_elements * 2];
    for (int i = 0; i < curr_element; i++)
        temp[i] = arr[i];
    arr = temp;
    max_elements *= 2;
}
```

Version 2

```
void resize() {
    delete [] arr;
    int *temp = new int[max_elements * 2];
    for (int i = 0; i < curr_element; i++)
        temp[i] = arr[i];
    arr = temp;
    max_elements *= 2;
}
```

Version 3

```
void resize() {
    int *temp = new int[max_elements * 2];
    for (int i = 0; i < curr_element; i++)
        temp[i] = arr[i];
    delete [] arr;
    arr = temp;
}
```

Answer

The correct order is: **allocate the new array, copy, delete the old array, re-point `arr`, update `max_elements`.**

Version 1: memory leak. The old array is never deleted, and after `arr = temp` nothing points to it.

- The output is correct (10 20 30 40 50 60), so the bug is invisible.
- Every resize loses the old block. Here the resizes go 2 to 4 to 8, which loses the 2-int block and the 4-int block: valgrind reports definitely lost: 24 bytes in 2 blocks (8 + 16).
- This is the lecture's "if missing the red head, space is not deallocated: memory leak".
- Fix: add `delete [] arr;` before `arr = temp;`.

Version 2: use after deallocation. The old array is deleted **before** its values are copied, so the loop reads freed memory.

- The first four values print as garbage that changes from run to run, while 50 and 60, added after the last resize, are correct. Two runs gave 1510118150 5 93220512 313892205 50 60 and 1511276467 5 898471892 1609121311 50 60.
- valgrind: Invalid read of size 4 ... 0 bytes inside a block of size 8 free'd, 6 errors in total (2 reads in the first resize and 4 in the second).
- Fix: copy first, then `delete [] arr;`.

Version 3: `max_elements` is not updated. The array really has 4 slots, but the object still believes the capacity is 2.

- After the first resize `curr_element` is 3, so `curr_element == max_elements` is never true again and no further resize happens.
- Adding the 5th and 6th elements writes **past the end of the heap block** (heap buffer overflow).
- The output still looks correct (10 20 30 40 50 60), but the heap is corrupted. valgrind: Invalid write of size 4 ... 0 bytes after a block of size 16 alloc'd (in `addAtEnd`) and Invalid read of size 4 (in `print`), 4 errors from 2 contexts.
- Fix: add `max_elements *= 2;`.

Correct version:

```
void IntVector::resize() {
    int *temp = new int[max_elements * 2];
    for (int i = 0; i < curr_element; i++)
        temp[i] = arr[i];
    delete [] arr;
    arr = temp;
    max_elements *= 2;
}
```

It prints 10 20 30 40 50 60, and valgrind reports no leaks and 0 errors.

Lesson: "it printed the right answer" does not prove the memory handling is right. Versions 1 and 3 both print correctly. None of the versions produces a compiler warning.

Verified: Dir verify_bank_oop/q14/. g++ -std=c++11 -Wall -Wextra -g -DVERSION=n bugs.cpp gave 0 warnings for every version. Each was run twice and under valgrind --leak-check=full:

- V0 (correct): printed 10 20 30 40 50 60. All heap blocks were freed, ERROR SUMMARY: 0 errors.
- V1: printed 10 20 30 40 50 60. 8 bytes in 1 blocks are definitely lost, 16 bytes in 1 blocks are definitely lost, definitely lost: 24 bytes in 2 blocks, ERROR SUMMARY: 2 errors from 2 contexts.
- V2: printed 1510118150 5 93220512 313892205 50 60, then 1511276467 5 898471892 1609121311 50 60. valgrind: Invalid read of size 4, Address 0x4e21080 is 0 bytes inside a block of size 8 free'd, ERROR SUMMARY: 6 errors from 1 contexts.
- V3: printed 10 20 30 40 50 60. valgrind: Invalid write of size 4 at 0x1094EE: IntVector::addAtEnd(int) (bugs.cpp:44), Address 0x4e210e0 is 0 bytes after a block of size 16 alloc'd, Invalid read of size 4 at 0x109536: IntVector::print(std::ostream&) (bugs.cpp:48), ERROR SUMMARY: 4 errors from 2 contexts.

OBJ-15 · output-prediction

(a) What does this program print? (b) What is the difference between **overloading** and **overriding**? Which one is used on each line? (c) What happens if `cout << compare(2.5, 1.5) << endl;` is added to `main`?

```
#include <iostream>
#include <string>
using namespace std;

int compare(int a, int b) {
    cout << "int ";
    return (a < b) ? -1 : (a > b) ? 1 : 0;
}

int compare(float a, float b) {
    cout << "float ";
    return (a < b) ? -1 : (a > b) ? 1 : 0;
}

class Person {
protected:
    string name;
public:
    Person(string n) : name(n) {}
    virtual ~Person() {}
    virtual void print() { cout << "Person " << name << endl; }
};

class Student : public Person {
public:
    Student(string n) : Person(n) {}
    void print() { cout << "Student " << name << endl; }
};

class Faculty : public Person {
public:
```

```

    Faculty(string n) : Person(n) {}
    void print() { cout << "Faculty " << name << endl; }
};

int main() {
    cout << compare(3, 4) << endl;           // 1
    cout << compare(2.5f, 1.5f) << endl;     // 2
    cout << compare(7, 7) << endl;          // 3
    Person *people[3] = { new Person("Pat"), new Student("Sam"), new Faculty("Fay") };
    for (int i = 0; i < 3; i++)
        people[i]->print();                 // 4, 5, 6
    for (int i = 0; i < 3; i++)
        delete people[i];
    return 0;
}

```

Answer

(a) Output

```

int -1
float 1
int 0
Person Pat
Student Sam
Faculty Fay

```

- In lines 1-3, `compare` runs (and prints its tag) **before** `cout` prints the returned value, because a function's arguments are evaluated before the function is called.
 - $3 < 4$ gives -1.
 - $2.5f > 1.5f$ gives 1, and the `f` suffix makes the arguments floats, so the float version runs.
 - $7 == 7$ gives 0.
- In lines 4-6 the array holds `Person*` values, but `print` is virtual, so each call runs the version of the **real object**.

(b)

- Overloading:** the **same name with different parameter lists** in the same scope, such as the lecture's `compare(int,int)` and `compare(float,float)`. The compiler chooses the version at **compile time** from the argument types (lines 1-3).
- Overriding:** a derived class redefines a base-class **virtual** method with the **same signature**, such as `Student::print()`. The version is chosen at **run time** from the object's real type through a base pointer or reference (lines 4-6). This is the polymorphism of the lecture's `Person *p = s; p->print()`.

(c) Compile error: call of overloaded '`compare(double, double)`' is ambiguous, with candidates `int compare(int, int)` and `int compare(float, float)`.

- 2.5 is a `double`, and there is no `compare(double, double)`.
- Converting double to int and double to float are equally ranked conversions, so the compiler cannot choose.
- Fix: call `compare(2.5f, 1.5f)`, cast the arguments (`compare((float)2.5, (float)1.5)` prints `float 1`), or add a `compare(double, double)` overload.

Verified: Dir verify_bank_oop/q15/.

- g++ -std=c++11 -Wall -Wextra -pedantic over.cpp -o over && ./over gave no warnings and printed:

```
int -1
float 1
int 0
Person Pat
Student Sam
Faculty Fay
```

- clang++ output was identical. valgrind: All heap blocks were freed, ERROR SUMMARY: 0 errors.
- -DBAD (adds compare(2.5, 1.5));

```
over.cpp:38:20: error: call of overloaded 'compare(double, double)' is ambiguous
over.cpp:5:5: note: candidate: 'int compare(int, int)'
over.cpp:9:5: note: candidate: 'int compare(float, float)'
```

- -DFIX (compare((float)2.5, (float)1.5)); that line printed float 1.